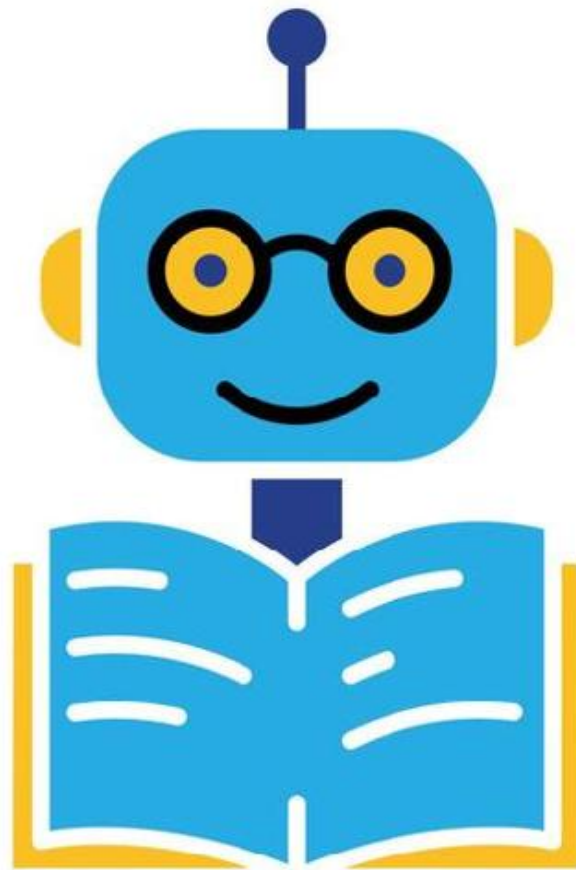


LD4019 – Algorithms and Programming Fundamentals

Campus Assistant Mini Project



Report By: Kristian Rossi

Student ID: 25011804

Contents

Introduction	4
Programme Requirements	4
Algorithms and Logic Used	4
Input Validation and Error Handling	4
Example 1 – study_rooms (list of dictionaries)	5
Example 2 – borrowed_items (list of dictionaries)	5
Example 3: events.json (list of dictionaries)	6
Example 4: Option/study_room_option – integer variables:	6
Algorithms & Flow Design	7
Pseudocode: Search for available rooms	7
Flow Chart: Search for available rooms	10
Pseudocode: Add borrowed items	13
Flow Chart: Add borrowed items	15
Task 3 & 4: Implementation & Testing Evidence	17
Programme Overview	17
Data Storage (JSON Files)	17
Python Code Explanation	22
Imports:	22
Formatting Variables:	22
Loading JSON data:	23
Saving data to JSON:	25
Backup and Restoration Functions:	26
Main programme loop	27
Initial programme run:	28
Main Programme Loop: Exit Option	30
Main Programme Loop: View Events Option	31
Main Programme Loop: Study Room Browsing and Searching	33
Main Programme Loop: Track, update and search borrowed items	45
Main Programme Loop: Backup/Restore Data	60
Validation of loop from start to end:	63
Critical Reflection	64

Ethical Considerations	64
Social Considerations	64
Security Considerations.....	64
References	65

Introduction

This report outlines the process of developing a Python-based Smart Campus Assistant to support students with a menu-driven interface. The application is built with Python and uses JSON files to store and access data persistently. The report has a structured layout, showing each stage of development including planning, designing, implementation and testing. Each section is explained and justified.

Programme Requirements

The application will be able to:

- Access data from JSON files
- View upcoming events
- View all study rooms and filter by booked and available rooms
- Add, remove, list and search for borrowed items
- Use validation rules to prevent programme crashes from user input
- Save updated/modified data back to JSON files

Algorithms and Logic Used

- If/elif/else for menu selection
- For loops to display data
- Boolean flags (e.g. found = False)
- List and dictionary searching using loops
- Conditional checks for uniqueness

Input Validation and Error Handling

- Try/except blocks for error handling/input validation Dates are validated using `datetime.strptime()`
- Status values are checked using if statements
- Prevents crashes and improves user experience

Task 2: Data Structures and Variables

Example 1 – study_rooms (list of dictionaries)

The study_rooms variable is implemented as a list of dictionaries, loaded from study_rooms.json. Each dictionary represents a single study room and stores attributes such as room ID, building, capacity, availability and available time range.

Dictionaries were used for individual rooms because they allow related attributes to be stored as key-value pairs, improving readability and direct access to specific fields.

The data structure supports the system's functionality by enabling filtering operations based on building, capacity, availability and time.

Example 2 – borrowed_items (list of dictionaries)

The borrowed_items variable is stored as a list of dictionaries, with each dictionary representing a borrowed item record.

A list has been used as it can store multiple items and easily be appended to/modified (essential for adding and removing borrowed items). This is essential for more scoped operations like filtering for certain keys and their values.

Example 3: events.json (list of dictionaries)

The events variable was implemented as a dictionary that contains a list of event records loaded from a JSON file.

The events data is stored as a dictionary containing a list of event records, allowing consistent access to event attributes while supporting iteration for display

Example 4: Option/study_room_option – integer variables:

The option and study_room_option variables are integers used to store the user's menu selections.

Integers are the most suitable for this because it makes the interface very simple for users and gives the terminal direct, simple and non-ambiguous input. This is very handy when using many options and sub option variables. The easy manipulation of integers makes error handling more efficient as there is less room for error, and the non-ambiguity of integer values makes overall algorithm processes seamless and straightforward.

Example 5: “found” – Boolean variable

The found variable is a Boolean flag used to track whether matching records have been identified during search operations.

Boolean is the most appropriate for this as it only has two values (True or False) which is exactly the value of data either being there (True) or not (False).

Algorithms & Flow Design

This section will provide the planning/design of two main functions of the smart assistant in the form of pseudocode and a flow chart.

Pseudocode: Search for available rooms

BEGIN SearchAvailableRooms

INPUT building_filter (may be blank)

INPUT capacity_raw (may be blank)

INPUT time_input (may be blank)

IF capacity_raw is not blank THEN

TRY convert capacity_raw to integer min_capacity

IF conversion demonstrated invalid THEN

OUTPUT "Invalid capacity"

RETURN to menu

ENDIF

ELSE

SET min_capacity = NONE

ENDIF

IF time_input is not blank THEN

TRY parse time_input as HH:MM to desired_time

IF parsing fails THEN

OUTPUT "Invalid time format"

RETURN to menu

ENDIF

ELSE

SET desired_time = NONE

ENDIF

SET found = FALSE

FOR each room in study_rooms DO

IF room.availability is not "Available" THEN

CONTINUE

ENDIF

IF building_filter not blank AND room.building != building_filter (case-insensitive) THEN

CONTINUE

ENDIF

IF min_capacity is not NONE AND room.capacity < min_capacity THEN

CONTINUE

ENDIF

IF desired_time is not NONE THEN

PARSE start_time from room.available_from

PARSE end_time from room.available_until

IF desired_time not within start_time to end_time THEN

CONTINUE

ENDIF

ENDIF

OUTPUT room details

SET found = TRUE

ENDFOR

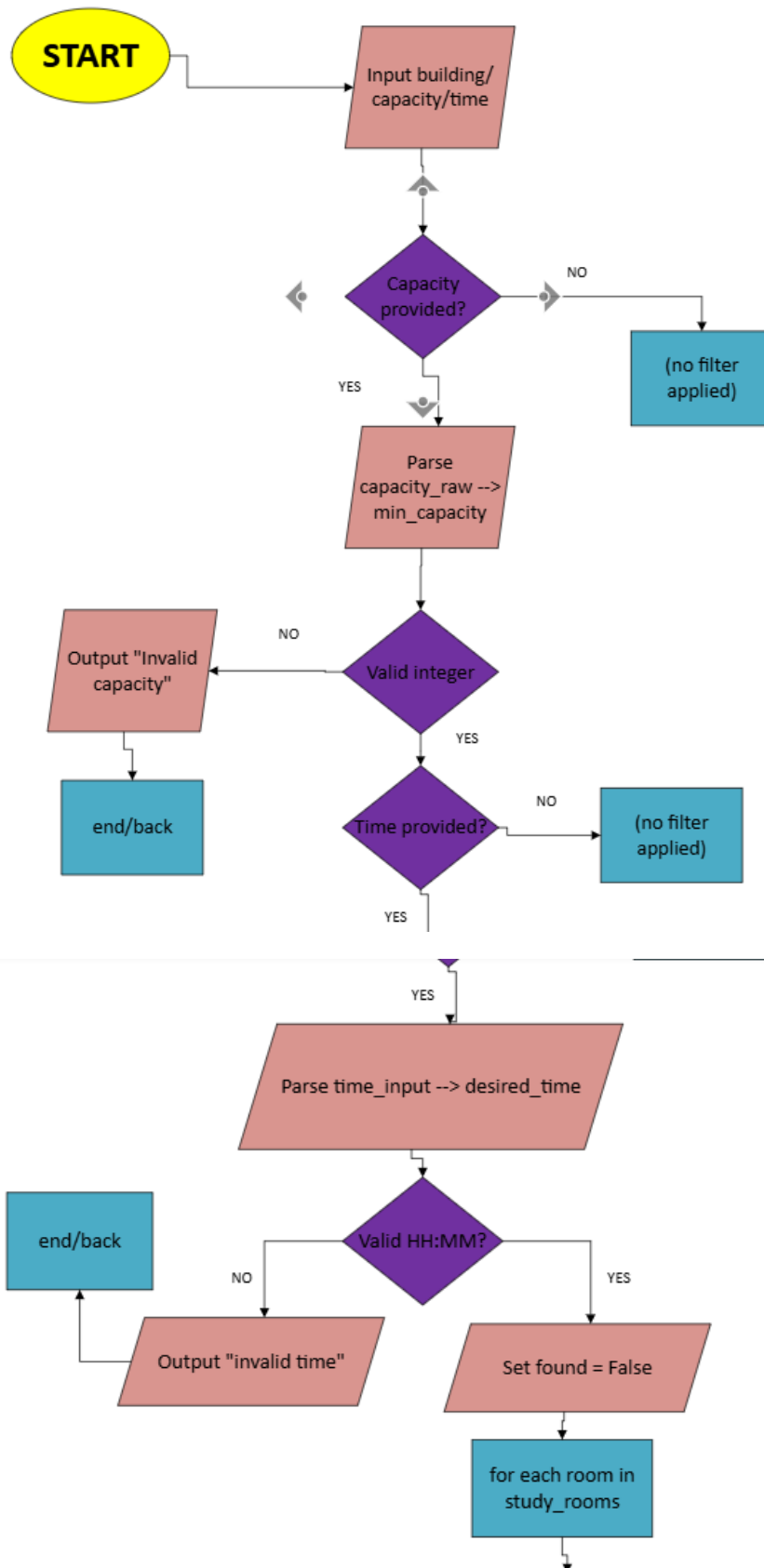
IF found is FALSE THEN

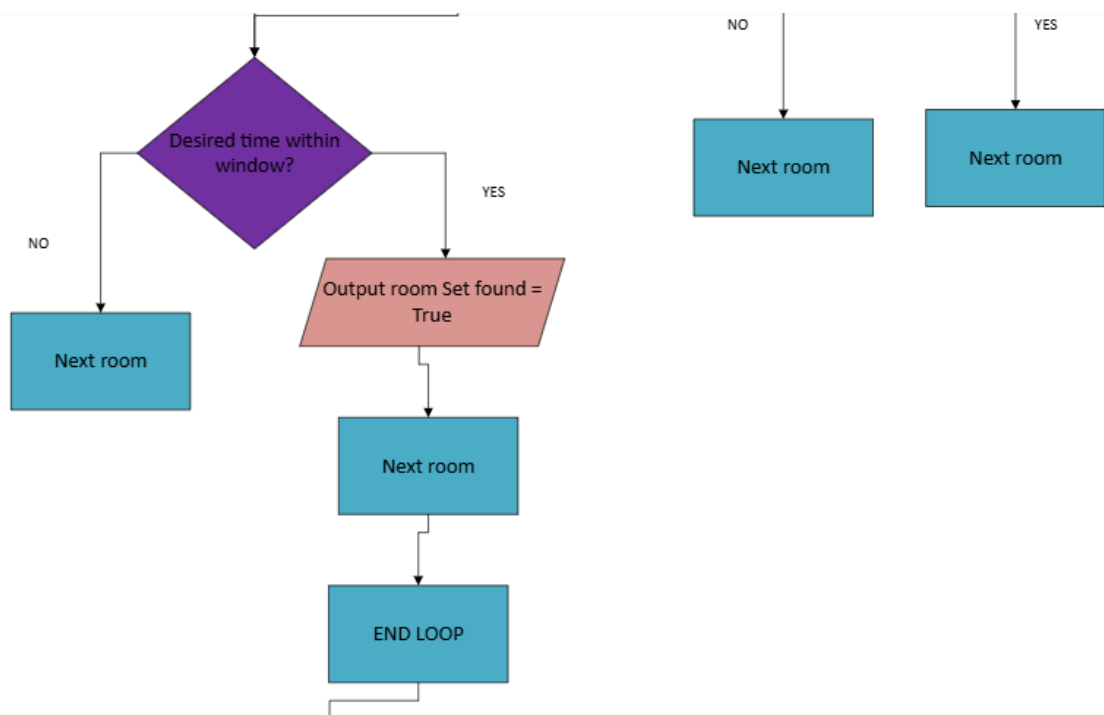
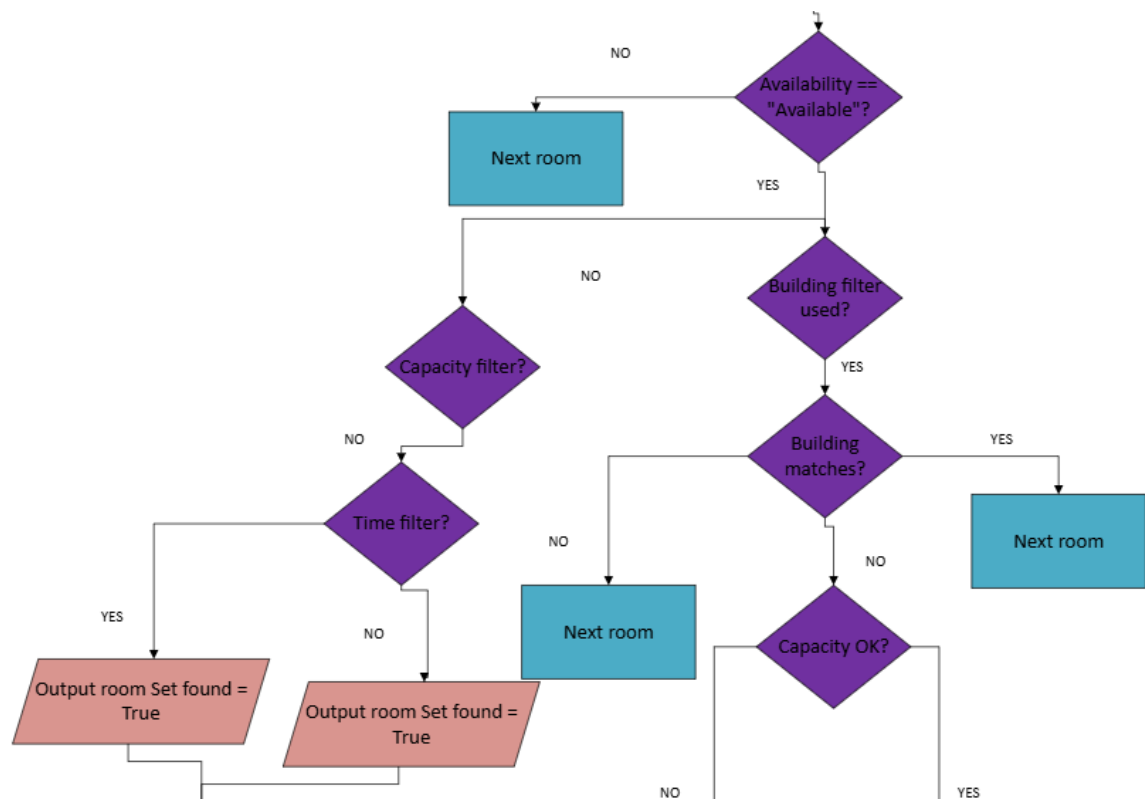
OUTPUT "No rooms matched criteria"

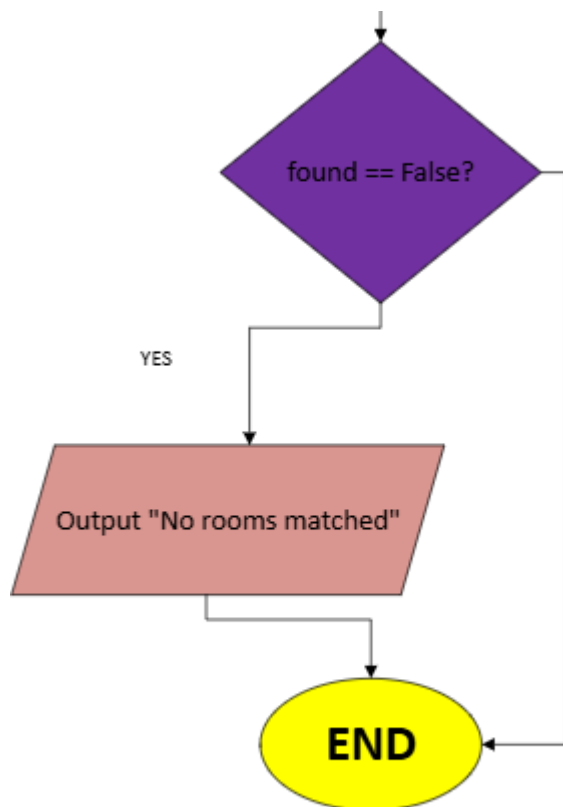
ENDIF

END

Flow Chart: Search for available rooms







This pseudocode and flowchart represent how the search available rooms function operates within the study room section works. They show that the user can apply zero, one or multiple optional filters to search for specific study room criteria stored in the study_rooms.json file.

Pseudocode: Add borrowed items

BEGIN AddBorrowedItem

INPUT item_id, item_name, student_id, student_name

REPEAT

INPUT borrow_date

TRY parse borrow_date as YYYY-MM-DD

IF invalid THEN OUTPUT error

UNTIL valid

REPEAT

INPUT due_date

TRY parse due_date as YYYY-MM-DD

IF invalid THEN OUTPUT error

UNTIL valid

REPEAT

INPUT status

IF status not in {"Borrowed", "Returned"} THEN OUTPUT error

UNTIL valid

IF any existing item in borrowed_items has same item_id THEN

OUTPUT "ID already exists"

RETURN to menu

ENDIF

CREATE new_item dictionary

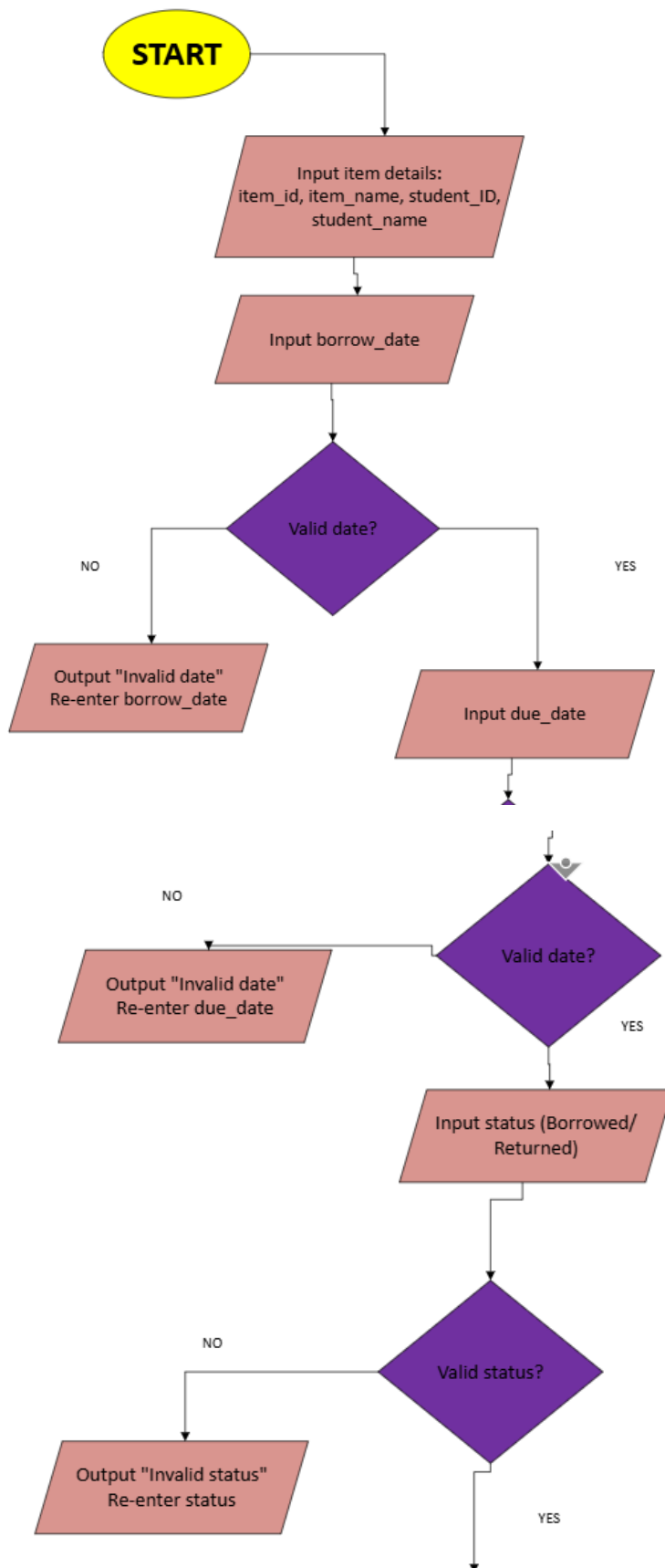
APP means append new_item to borrowed_items list

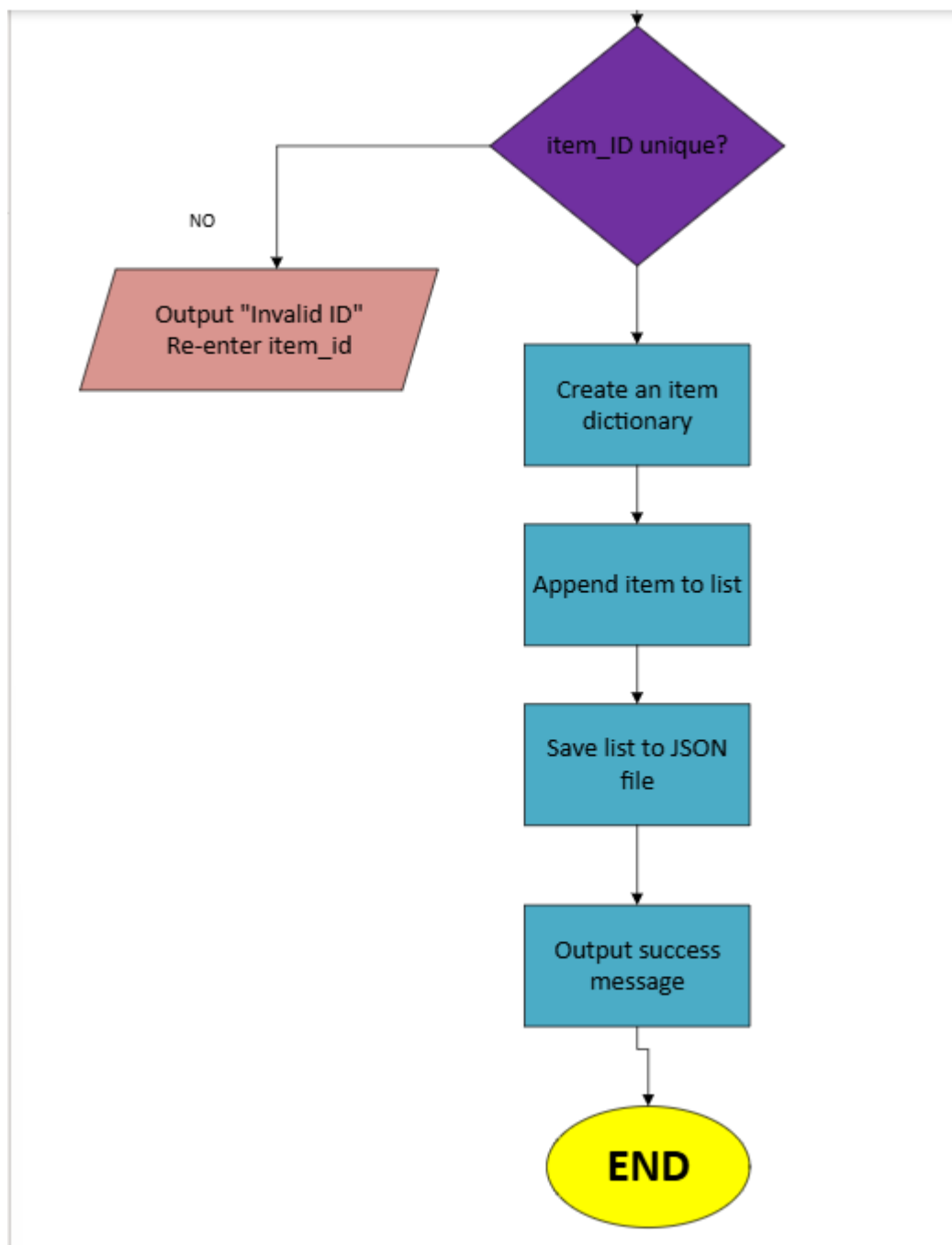
SAVE borrowed_items to JSON

OUTPUT "Borrowed item added successfully"

END

Flow Chart: Add borrowed items





The pseudocode and flow chart illustrate the adding borrowed items function by showing how user input is collected, validated and checked for duplicate item IDs before a new record is created and saved to the JSON file. This ensures data integrity and persistent storage of borrowed item records.

Task 3 & 4: Implementation & Testing Evidence

Programme Overview

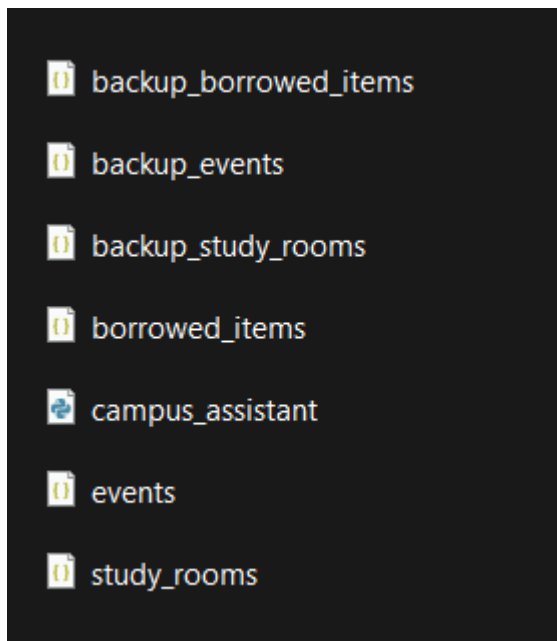


Figure 1 - Project folder, including the main py file and json files.

Data Storage (JSON Files)

- The program uses three JSON files:
- events.json
- study_rooms.json
- borrowed_items.json
- backup_events.json
- backup_study_rooms.json
- backup_borrowed_items.json

Json file was chosen for data storage because it is lightweight, human readable and easy to load and save using Python.

```
{ } borrowed_items.json X
{ } borrowed_items.json > { } 1
1  [
2      {
3          "item_id": "B001",
4          "item_name": "Laptop",
5          "student_id": "ST1001",
6          "student_name": "Alice Green",
7          "borrow_date": "2025-03-01",
8          "due_date": "2025-03-10",
9          "status": "Borrowed"
10     },
11     {
12         "item_id": "B002",
13         "item_name": "Microscope",
14         "student_id": "ST1002",
15         "student_name": "James Brown",
16         "borrow_date": "2025-03-03",
17         "due_date": "2025-03-12",
18         "status": "Borrowed"
19     },
20     {
21         "item_id": "B003",
22         "item_name": "Football Kit",
23         "student_id": "ST1003",
24         "student_name": "Sarah Lee",
25         "borrow_date": "2025-03-05",
26         "due_date": "2025-03-15",
27         "status": "Returned"
28     }
29 ]
```

Figure 2 – borrowed_items.json detail.

```
{ } events.json X

{ } events.json > [ ] events > { } 1
1  {"events":[
2    {
3      "event_id": "E001",
4      "event_name": "AI Research Seminar",
5      "date": "2025-03-12",
6      "time": "10:00",
7      "location": "Main Auditorium"
8    },
9    {
10     "event_id": "E002",
11     "event_name": "Data Science Workshop",
12     "date": "2025-03-15",
13     "time": "14:00",
14     "location": "Lab 204"
15   },
16   {
17     "event_id": "E003",
18     "event_name": "Student Career Fair",
19     "date": "2025-03-20",
20     "time": "09:30",
21     "location": "Sports Hall"
22   },
23   {
24     "event_id": "E004",
25     "event_name": "Cybersecurity Awareness Talk",
26     "date": "2025-03-22",
27     "time": "11:00",
28     "location": "Room B101"
29   },
30   {
31     "event_id": "E005",
32     "event_name": "Sustainability Expo",
33     "date": "2025-03-25",
34     "time": "13:00",
35     "location": "Campus Green"
36   }
37 ] }
```

Figure 3 – events.json detail.

```
{ } study_rooms.json ×
{ } study_rooms.json > { } 4
1  ✓ [
2  ✓ {
3      "room_id": "R001",
4      "building": "Library",
5      "capacity": 4,
6      "availability": "Available",
7      "available_from": "09:00",
8      "available_until": "18:00"
9  },
10 ✓ {
11     "room_id": "R002",
12     "building": "Library",
13     "capacity": 6,
14     "availability": "Booked",
15     "available_from": "09:00",
16     "available_until": "18:00"
17 },
18 ✓ {
19     "room_id": "R003",
20     "building": "Engineering Block",
21     "capacity": 10,
22     "availability": "Available",
23     "available_from": "14:00",
24     "available_until": "18:00"
25 },
26 ✓ {
27     "room_id": "R004",
28     "building": "Science Building",
29     "capacity": 8,
30     "availability": "Available",
31     "available_from": "13:00",
32     "available_until": "20:00"
33 },
```

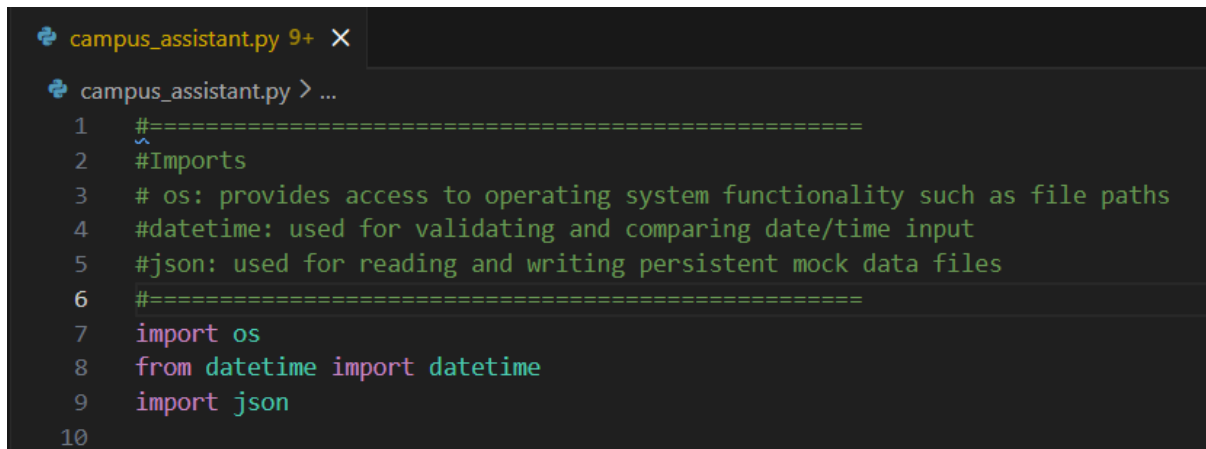
Figure 4 - study_rooms.json detail (section 1).

```
{ } study_rooms.json X
{ } study_rooms.json > { } 4
33 {
34 {
35   "room_id": "R005",
36   "building": "Arts Centre",
37   "capacity": 5,
38   "availability": "Booked",
39   "available_from": "12:00",
40   "available_until": "18:00"
41 }
42 ]
43
```

Figure 5 - study_rooms.json detail (section 2).

Python Code Explanation

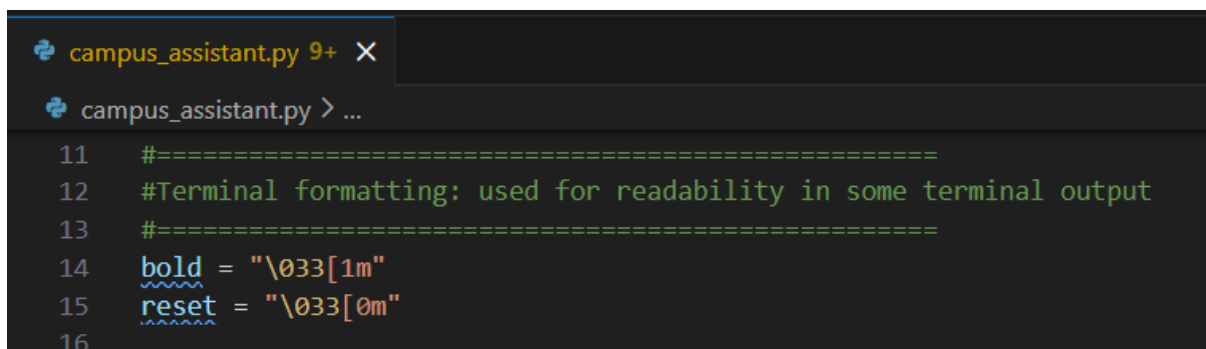
Imports:

A screenshot of a code editor window titled 'campus_assistant.py 9+ X'. The editor shows the following Python code:

```
1  #=====
2  #Imports
3  # os: provides access to operating system functionality such as file paths
4  #datetime: used for validating and comparing date/time input
5  #json: used for reading and writing persistent mock data files
6  #=====
7  import os
8  from datetime import datetime
9  import json
10
```

Figure 6 - This shows the imports used in the python file (json and datetime)

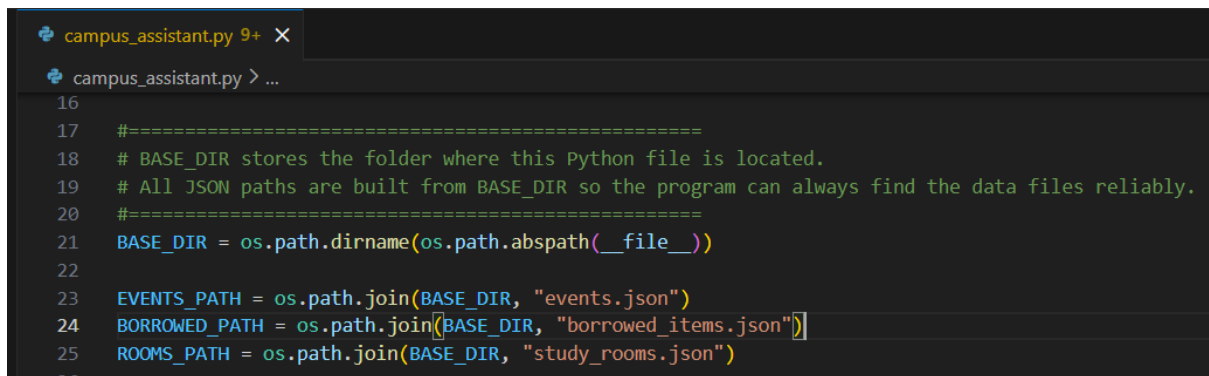
Formatting Variables:

A screenshot of a code editor window titled 'campus_assistant.py 9+ X'. The editor shows the following Python code:

```
11  #=====
12  #Terminal formatting: used for readability in some terminal output
13  #=====
14  bold = "\033[1m"
15  reset = "\033[0m"
16
```

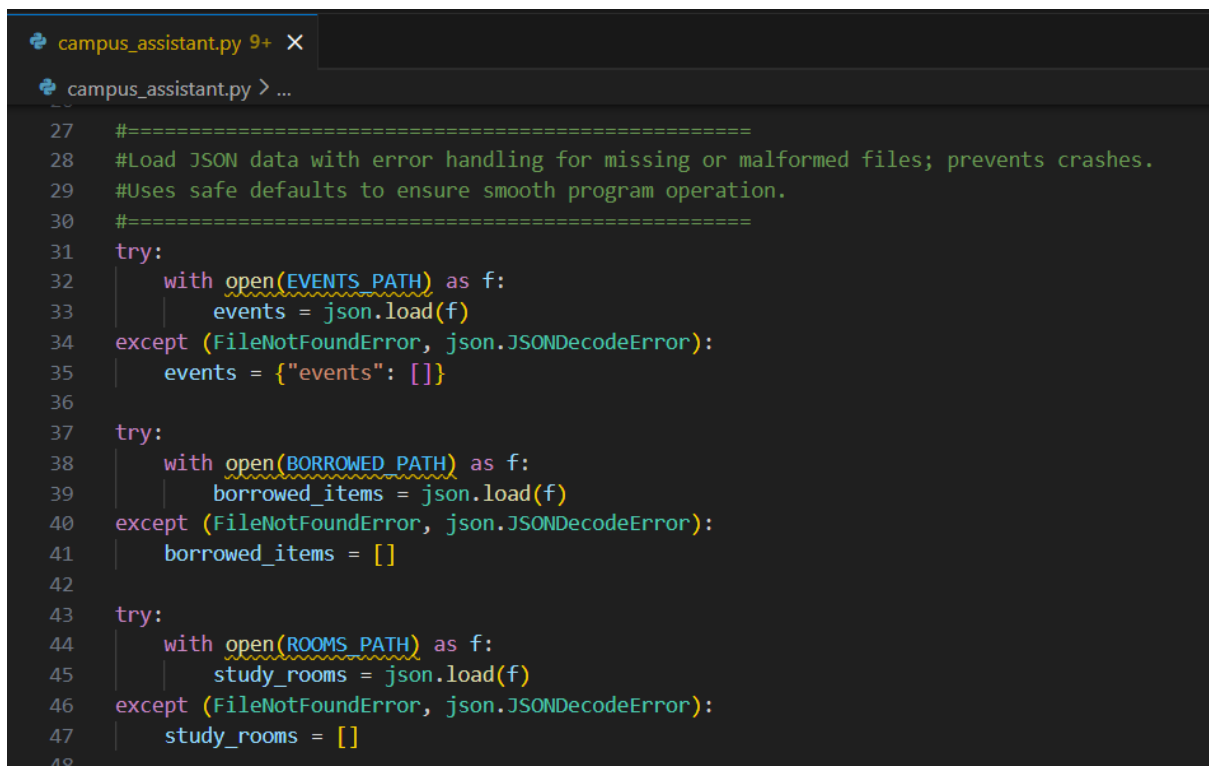
Figure 7 - These variables store ANSI escape codes, used to format text displayed in the terminal.

Loading JSON data:



```
campus_assistant.py 9+ X
campus_assistant.py > ...
16
17 #=====
18 # BASE_DIR stores the folder where this Python file is located.
19 # All JSON paths are built from BASE_DIR so the program can always find the data files reliably.
20 #=====
21 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
22
23 EVENTS_PATH = os.path.join(BASE_DIR, "events.json")
24 BORROWED_PATH = os.path.join(BASE_DIR, "borrowed_items.json")
25 ROOMS_PATH = os.path.join(BASE_DIR, "study_rooms.json")
26
```

Figure 8 - This screenshot shows how the program uses `BASE_DIR` and `os.path.join()` to build absolute paths for all JSON data files, ensuring reliable file access across different environments.



```
campus_assistant.py 9+ X
campus_assistant.py > ...
27 #=====
28 #Load JSON data with error handling for missing or malformed files; prevents crashes.
29 #Uses safe defaults to ensure smooth program operation.
30 #=====
31 try:
32     with open(EVENTS_PATH) as f:
33         events = json.load(f)
34 except (FileNotFoundError, json.JSONDecodeError):
35     events = {"events": []}
36
37 try:
38     with open(BORROWED_PATH) as f:
39         borrowed_items = json.load(f)
40 except (FileNotFoundError, json.JSONDecodeError):
41     borrowed_items = []
42
43 try:
44     with open(ROOMS_PATH) as f:
45         study_rooms = json.load(f)
46 except (FileNotFoundError, json.JSONDecodeError):
47     study_rooms = []
48
```

Figure 9 - This code block opens and reads data from external JSON files using try/except blocks.

Test 1: The programme will not crash if it cannot find the JSON file

```
EXPLORER
LD4019 - ASSIGNMENT - PY
  backup_borrowed_items.json
  backup_events.json
  backup_study_rooms.json
  borrowed_items.json
  campus_assistant.py 9+
  events_old.json
  study_rooms.json

campus_assistant.py 9+ X {} events_old.json

campus_assistant.py > ...
407
408 elif option == 4:
409     action = input("Enter B to backup or R to
410     if action == "b":
411         backup_all_data()
412     elif action == "r":
413         restore_all_data()
414     else:
415         print("Invalid option.")
416
417
418 else:

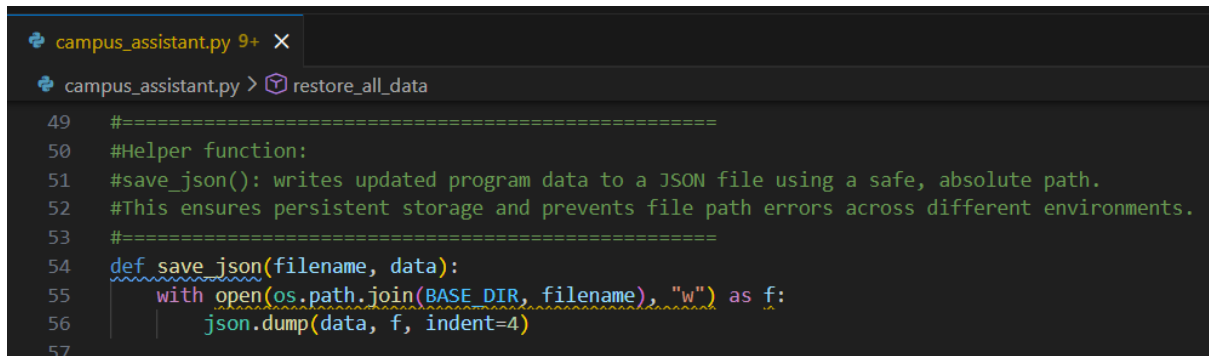
PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\krist\OneDrive - Northumbria University - Product
thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumb
"

Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
1
Upcoming Events:
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
```

Figure 10 - This shows that the events.json file name has been changed, meaning the programme will not be able to find events.json. The programme cannot reach the events file and keeps running without crashing. When option 1 is selected, no events are displayed on the terminal, and the user is returned to the main menu.

Saving data to JSON:



```
campus_assistant.py 9+ X
campus_assistant.py > restore_all_data

49  #=====
50  #Helper function:
51  #save_json(): writes updated program data to a JSON file using a safe, absolute path.
52  #This ensures persistent storage and prevents file path errors across different environments.
53  #=====
54  def save_json(filename, data):
55      with open(os.path.join(BASE_DIR, filename), "w") as f:
56          json.dump(data, f, indent=4)
57
```

Figure 11 - This function is responsible for saving updated program data back to its corresponding JSON file.

Backup and Restoration Functions:

```
campus_assistant.py 9+ X
campus_assistant.py > ...

58 #=====
59 #Backup/Restore Functions
60 #backup_all_data(): saves all datasets to separate backup JSON files
61 #restore_all_data(): loads backup JSON files back into memory (overwrites current data)
62 #=====
63 def backup_all_data():
64     save_json("backup_events.json", events)
65     save_json("backup_study_rooms.json", study_rooms)
66     save_json("backup_borrowed_items.json", borrowed_items)
67     print("All data backed up successfully.")
68
69 def restore_all_data():
70     global events, study_rooms, borrowed_items
71     try:
72         with open(os.path.join(BASE_DIR, "backup_events.json")) as f:
73             events = json.load(f)
74         with open(os.path.join(BASE_DIR, "backup_study_rooms.json")) as f:
75             study_rooms = json.load(f)
76         with open(os.path.join(BASE_DIR, "backup_borrowed_items.json")) as f:
77             borrowed_items = json.load(f)
78         print("All data restored successfully.")
79     except FileNotFoundError:
80         print("Backup files not found. Restore failed.")
81
82
```

Figure 12 - Backup and restore functions for JSON data files, enabling safe data recovery and persistence.

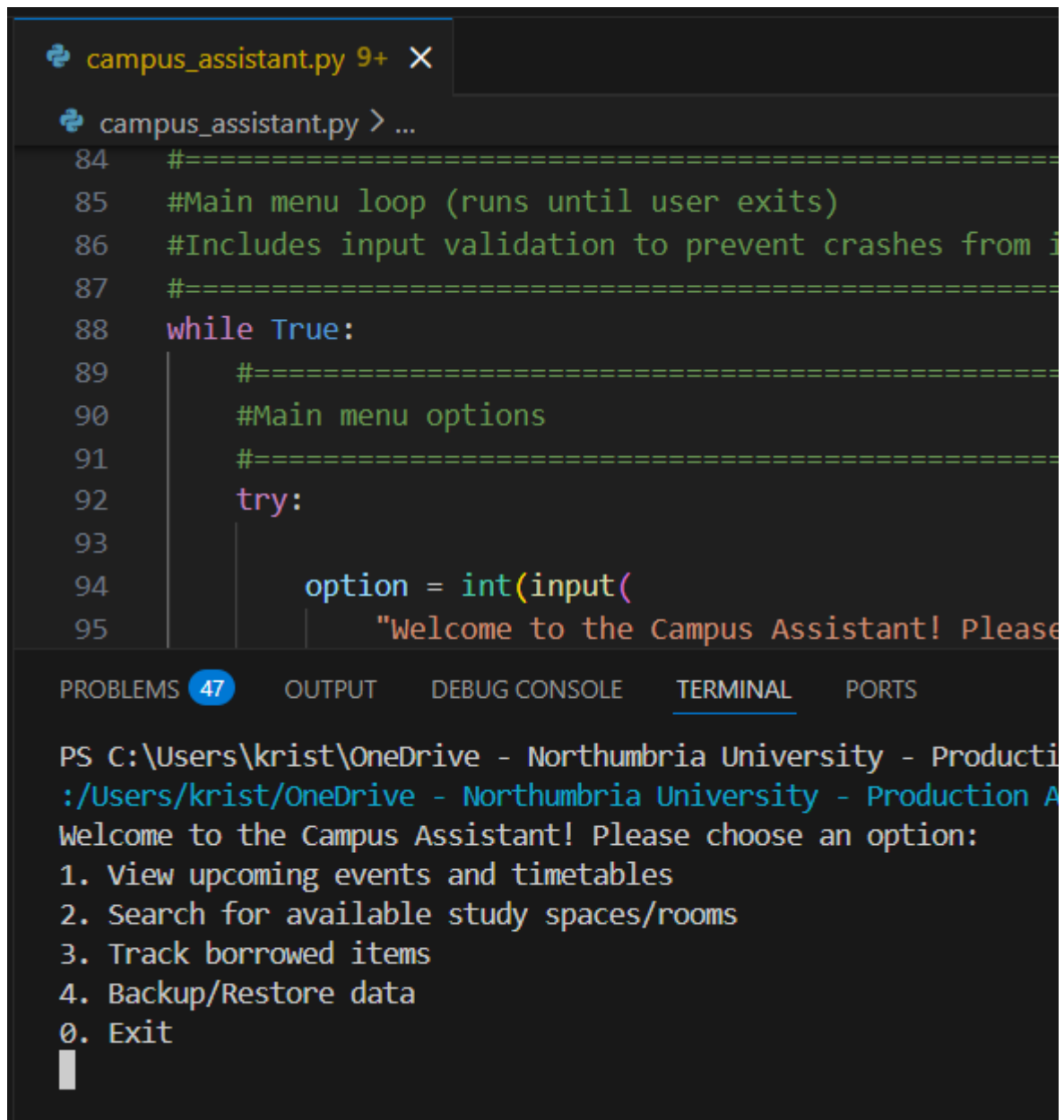
Main programme loop

Main Programme Loop: Main Menu Options

```
campus_assistant.py 9+ X
campus_assistant.py > restore_all_data
84  #=====
85  #Main menu loop (runs until user exits)
86  #Includes input validation to prevent crashes from invalid inputs.
87  #=====
88  while True:
89      #=====
90      #Main menu options
91      #=====
92      try:
93
94          option = int(input(
95              "Welcome to the Campus Assistant! Please choose an option:\n"
96              "1. View upcoming events and timetables\n"
97              "2. Search for available study spaces/rooms\n"
98              "3. Track borrowed items\n"
99              "4. Backup/Restore data\n"
100             "0. Exit\n"
101             ).strip())
102
103     except ValueError:
104         print("Invalid input. Please enter a number corresponding to the options.")
105         continue
106
```

Figure 13 - This block of code creates the initial main menu screen on running the programme.

Test 2: Initial programme run



The image shows a code editor window with a file named `campus_assistant.py` open. The code is written in Python and includes several comments and a `while` loop. The terminal window below the code editor shows the output of the program, which is a welcome message and a list of options for the user to choose from.

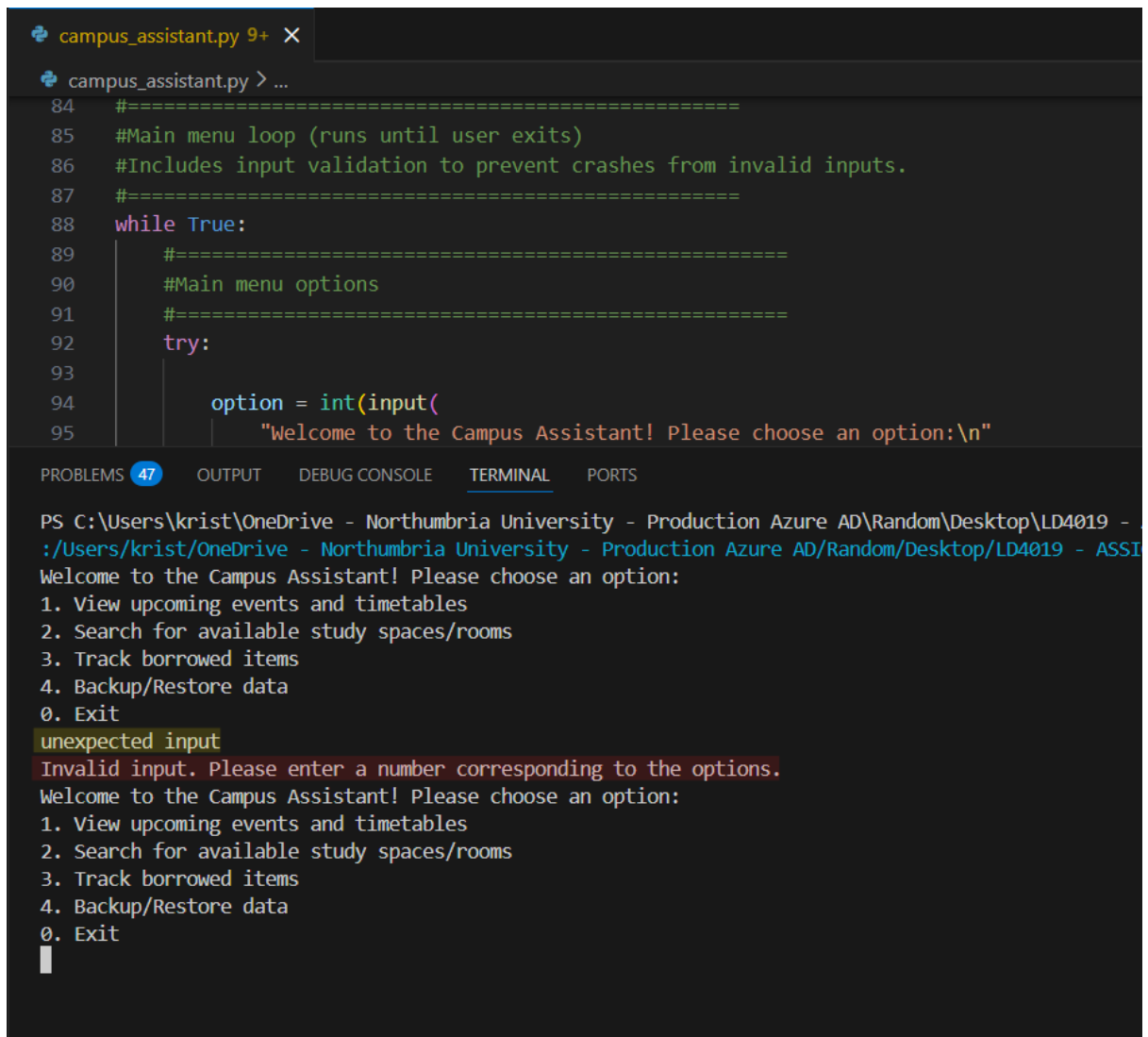
```
campus_assistant.py 9+ X
campus_assistant.py > ...
84  #=====
85  #Main menu loop (runs until user exits)
86  #Includes input validation to prevent crashes from i
87  #=====
88  while True:
89      #=====
90      #Main menu options
91      #=====
92      try:
93
94          option = int(input(
95              "Welcome to the Campus Assistant! Please
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\krist\OneDrive - Northumbria University - Producti
:/Users/krist/OneDrive - Northumbria University - Production A
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
█
```

Figure 14 - This is the first output that shows up when the programme is initially run. Shows the main menu is implemented and instantly prompted on running.

Test 3: Input Validation



The screenshot displays a code editor with a file named `campus_assistant.py`. The code is a Python script for a campus assistant application. It features a main menu loop that runs until the user exits. The loop includes input validation to prevent crashes from invalid inputs. The code is as follows:

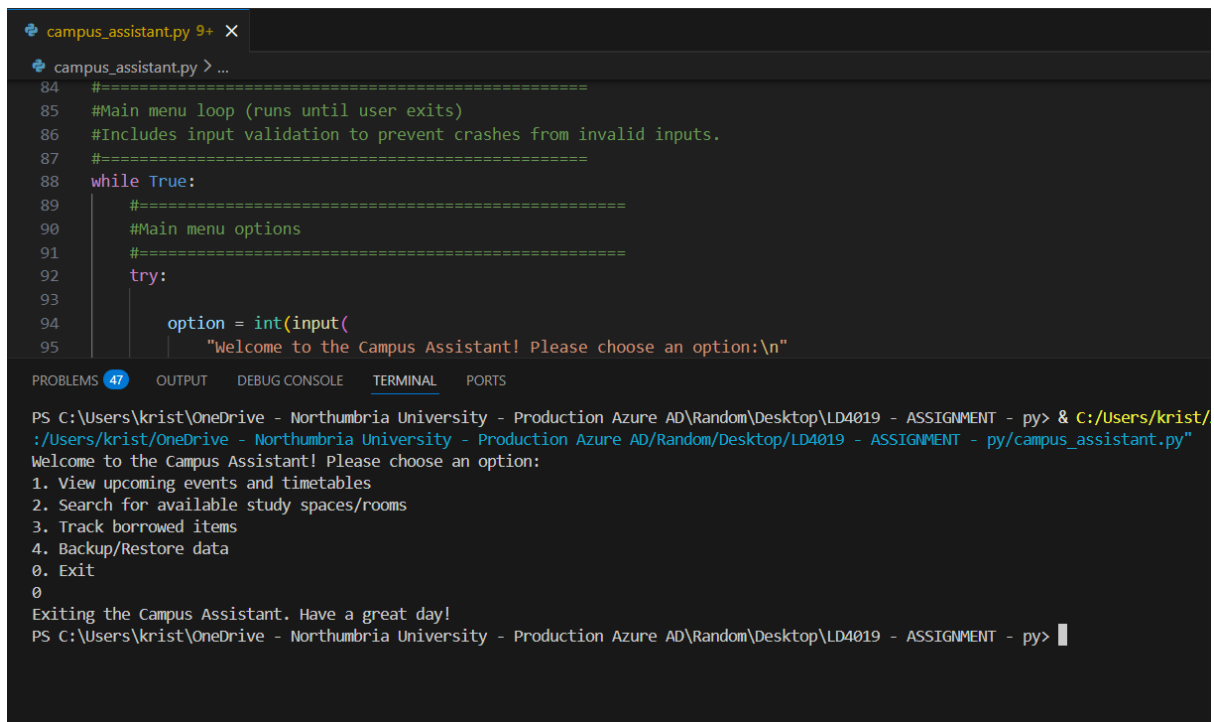
```
84 #=====
85 #Main menu loop (runs until user exits)
86 #Includes input validation to prevent crashes from invalid inputs.
87 #=====
88 while True:
89     #=====
90     #Main menu options
91     #=====
92     try:
93
94         option = int(input(
95             "Welcome to the Campus Assistant! Please choose an option:\n"
```

The terminal output shows the execution of the program. It displays the welcome message and the main menu options:

```
PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random\Desktop\LD4019 - .
:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - ASSI
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
unexpected input
Invalid input. Please enter a number corresponding to the options.
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
```

Figure 15 - This shows that an error message occurred when an invalid input was detected and the application asks the user to enter a valid input.

Test 4: The programme responds to input

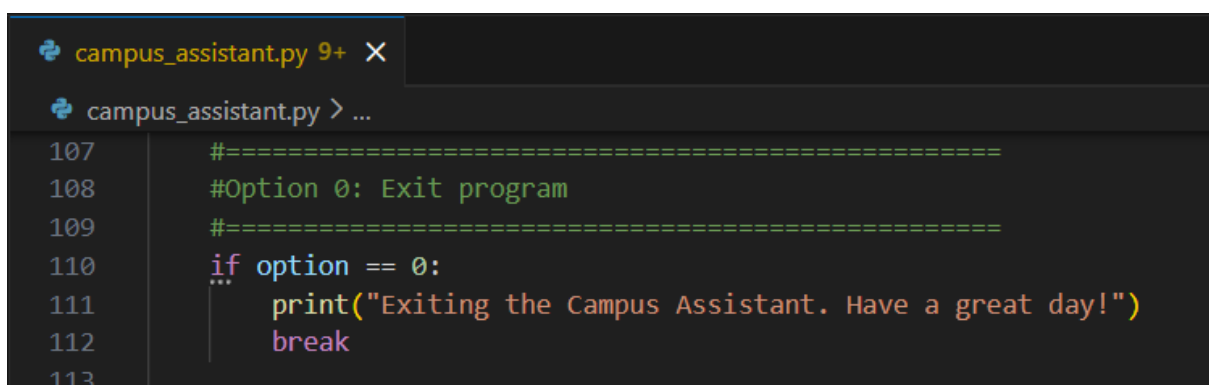


```
campus_assistant.py 9+ X
campus_assistant.py > ...
84 #=====
85 #Main menu loop (runs until user exits)
86 #Includes input validation to prevent crashes from invalid inputs.
87 #=====
88 while True:
89     #=====
90     #Main menu options
91     #=====
92     try:
93
94         option = int(input(
95             "Welcome to the Campus Assistant! Please choose an option:\n"

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random\Desktop\LD4019 - ASSIGNMENT - py> & C:/Users/krist/
:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - ASSIGNMENT - py/campus_assistant.py"
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
0
Exiting the Campus Assistant. Have a great day!
PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random\Desktop\LD4019 - ASSIGNMENT - py> █
```

Figure 16 - This shows the application responding to a valid input (the exit option).

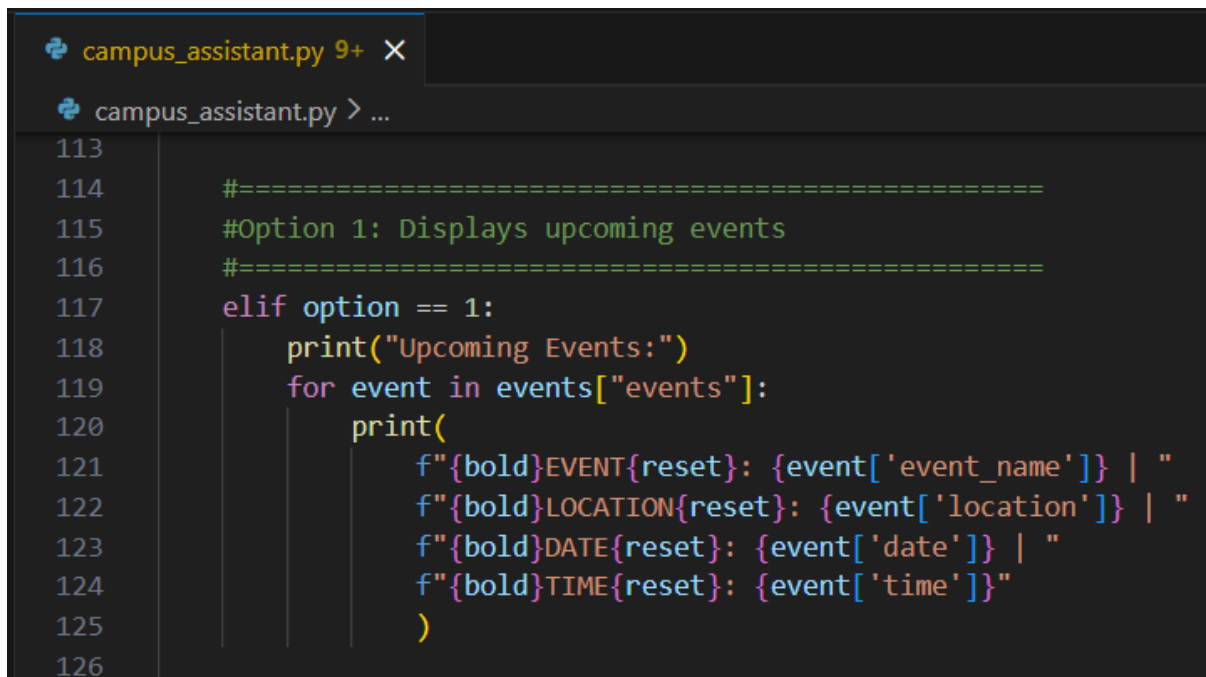
Main Programme Loop: Exit Option



```
campus_assistant.py 9+ X
campus_assistant.py > ...
107 #=====
108 #Option 0: Exit program
109 #=====
110 if option == 0:
111     print("Exiting the Campus Assistant. Have a great day!")
112     break
113
```

Figure 17 - This block of code controls the output of the user choosing option 0 (Exit).

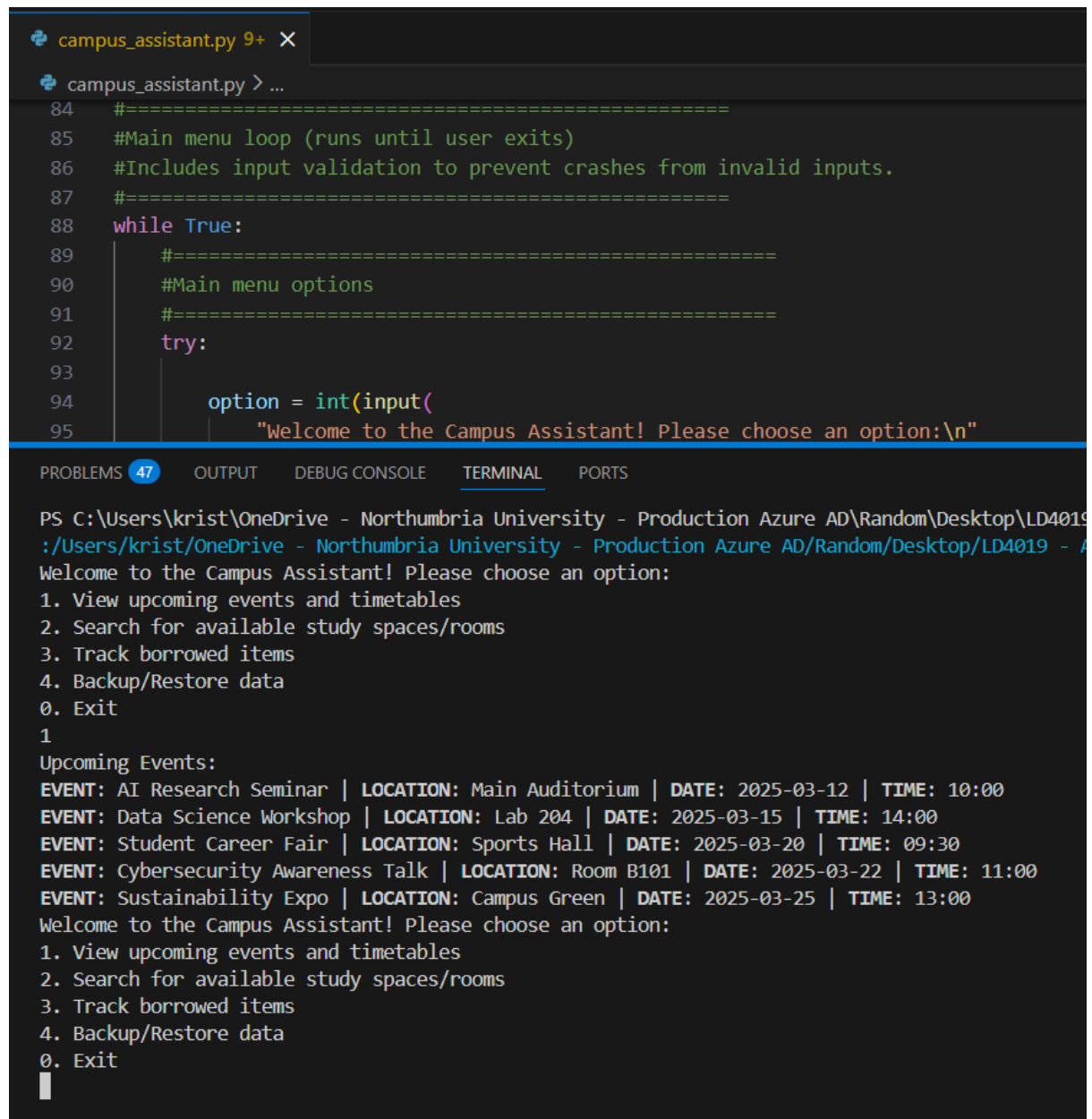
Main Programme Loop: View Events Option



```
campus_assistant.py 9+ X
campus_assistant.py > ...
113
114     #=====
115     #Option 1: Displays upcoming events
116     #=====
117     elif option == 1:
118         print("Upcoming Events:")
119         for event in events["events"]:
120             print(
121                 f"{bold}EVENT{reset}: {event['event_name']} | "
122                 f"{bold}LOCATION{reset}: {event['location']} | "
123                 f"{bold}DATE{reset}: {event['date']} | "
124                 f"{bold}TIME{reset}: {event['time']}"
125             )
126
```

Figure 18 - This shows the view events section of the loop. (option1)

Test 5: Option 1 displays all events in the terminal



The image shows a VS Code editor window with a file named `campus_assistant.py`. The code is a Python script for a campus assistant. It features a main menu loop that prompts the user to choose an option. The options are: 1. View upcoming events and timetables, 2. Search for available study spaces/rooms, 3. Track borrowed items, 4. Backup/Restore data, and 0. Exit. The script uses a `while True` loop and a `try` block to handle user input. The terminal output shows the program running in a PowerShell prompt, displaying the welcome message and the menu options. The user has entered '1', and the program has displayed a list of upcoming events with their locations, dates, and times.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
84 #=====
85 #Main menu loop (runs until user exits)
86 #Includes input validation to prevent crashes from invalid inputs.
87 #=====
88 while True:
89     #=====
90     #Main menu options
91     #=====
92     try:
93
94         option = int(input(
95             "Welcome to the Campus Assistant! Please choose an option:\n"
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random\Desktop\LD4019
:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - A
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
1
Upcoming Events:
EVENT: AI Research Seminar | LOCATION: Main Auditorium | DATE: 2025-03-12 | TIME: 10:00
EVENT: Data Science Workshop | LOCATION: Lab 204 | DATE: 2025-03-15 | TIME: 14:00
EVENT: Student Career Fair | LOCATION: Sports Hall | DATE: 2025-03-20 | TIME: 09:30
EVENT: Cybersecurity Awareness Talk | LOCATION: Room B101 | DATE: 2025-03-22 | TIME: 11:00
EVENT: Sustainability Expo | LOCATION: Campus Green | DATE: 2025-03-25 | TIME: 13:00
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
█

Figure 19 - This shows the output of the user selecting the view events option (option 1)

Main Programme Loop: Study Room Browsing and Searching

```
campus_assistant.py 9+ X
campus_assistant.py > ...

126
127 #=====
128 #Option 2: Study room browsing and searching
129 #Includes filtering by building, capacity, and time.
130 #=====
131 elif option == 2:
132     while True:
133         try:
134             study_room_option = int(input(
135                 "Would you like to:\n"
136                 "1. View all study rooms\n"
137                 "2. View all available study rooms\n"
138                 "3. View all booked study rooms\n"
139                 "4. Search for available rooms (filters: building/capacity/time)\n"
140                 "0. Back to main menu\n"
141             ).strip())
142         except ValueError:
143             print("Invalid input. Please enter a number corresponding to the options.")
144             continue
145
146         if study_room_option == 0:
147             break
148
149         elif study_room_option == 1:
150             print("\nAll Study Rooms:\n")
151             for room in study_rooms:
152                 print(
153                     f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
154                     f"{bold}BUILDING{reset}: {room.get('building')} | "
155                     f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
156                     f"{bold}AVAILABILITY{reset}: {room.get('availability')} | "
157                     f"{bold}FROM{reset}: {room.get('available_from')} | "
158                     f"{bold}UNTIL{reset}: {room.get('available_until')}"
159                 )
160
```

Figure 20 - Study room code (section 1).

```
campus_assistant.py 9+ X
campus_assistant.py > ...

160
161 elif study_room_option == 2:
162     print("\nAvailable Study Rooms:\n")
163     found = False
164     for room in study_rooms:
165         if room.get("availability") == "Available":
166             print(
167                 f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
168                 f"{bold}BUILDING{reset}: {room.get('building')} | "
169                 f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
170                 f"{bold}AVAILABILITY{reset}: {room.get('availability')} | "
171                 f"{bold}FROM{reset}: {room.get('available_from')} | "
172                 f"{bold}UNTIL{reset}: {room.get('available_until')}"
173             )
174             found = True
175     if not found:
176         print("No available study rooms found.")
177
178 elif study_room_option == 3:
179     print("\nBooked Study Rooms:\n")
180     found = False
181     for room in study_rooms:
182         if room.get("availability") == "Booked":
183             print(
184                 f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
185                 f"{bold}BUILDING{reset}: {room.get('building')} | "
186                 f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
187                 f"{bold}AVAILABILITY{reset}: {room.get('availability')} | "
188                 f"{bold}FROM{reset}: {room.get('available_from')} | "
189                 f"{bold}UNTIL{reset}: {room.get('available_until')}"
190             )
191             found = True
192     if not found:
193         print("No booked study rooms found.")
```

Figure 21 - Study room code (section 2).

```
campus_assistant.py 9+ x
campus_assistant.py > ...
194 #=====
195 #Search available rooms using sequential filtering:
196 #1) availability must be "Available"
197 #2) building matches (if provided)
198 #3) capacity meets minimum (if provided)
199 #4) desired time falls within available_from .. available_until .. (if provided)
200 #=====
201 elif study_room_option == 4:
202
203     building_filter = input("Which building would you like to study in? (press Enter to skip): ").strip()
204     capacity_raw = input("What is the minimum capacity you require? (press Enter to skip): ").strip()
205     time_input = input("Enter desired time (HH:MM) (press Enter to skip): ").strip()
206
207     # Capacity validation
208     min_capacity = None
209     if capacity_raw:
210         try:
211             min_capacity = int(capacity_raw)
212         except ValueError:
213             print("Invalid capacity input. Please enter a number.")
214             continue
215
216     # Time validation (single time, not a range)
217     desired_time = None
218     if time_input:
219         try:
220             desired_time = datetime.strptime(time_input, "%H:%M").time()
221         except ValueError:
222             print("Invalid time format. Please use HH:MM (e.g., 14:30).")
223             continue
224
225     found = False
226
```

Figure 22 - Study room code (section 3).

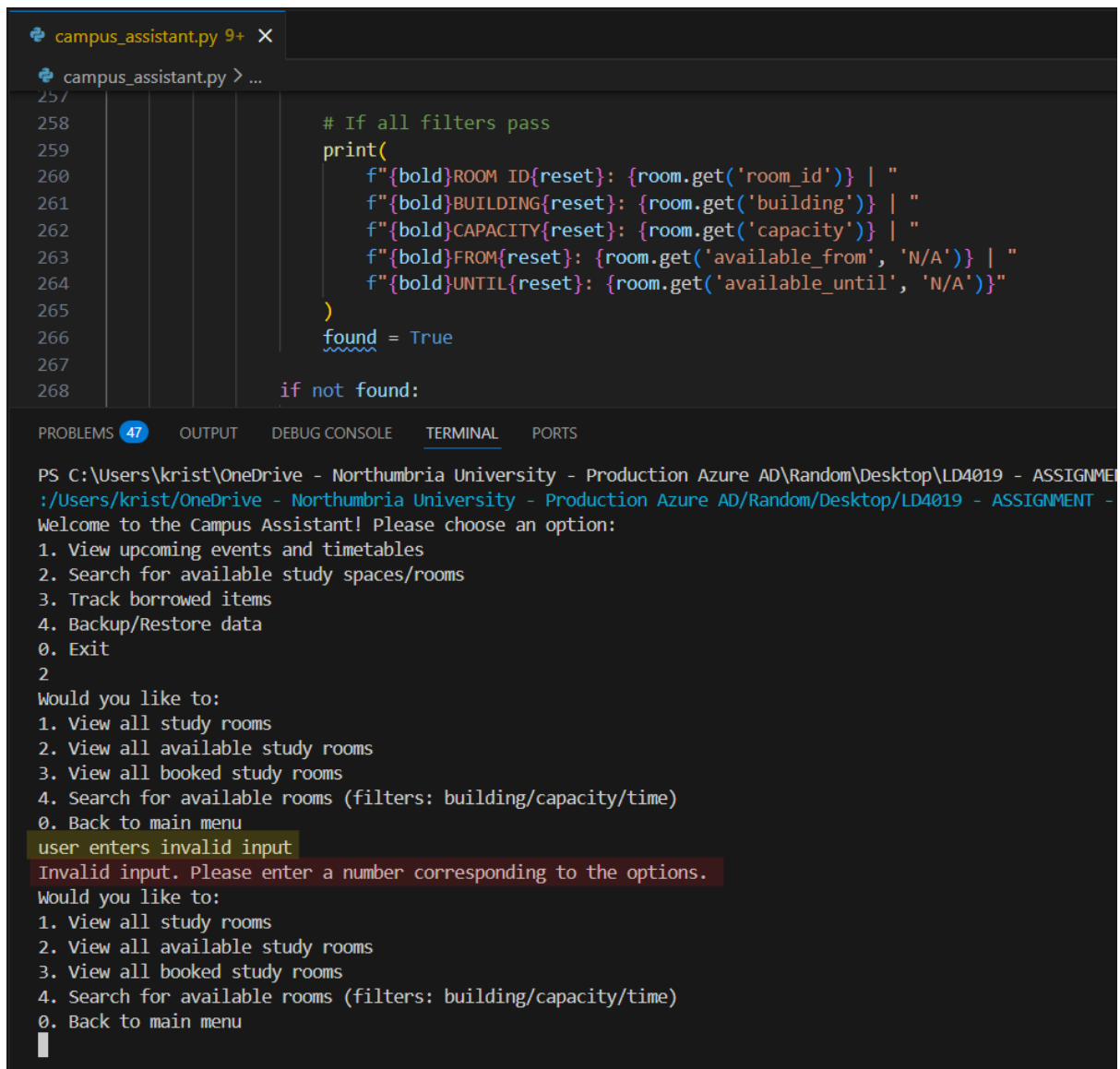
```
campus_assistant.py 9+ X
campus_assistant.py > ...
227     for room in study_rooms:
228         # Only AVAILABLE rooms (matches the requirement)
229         if room.get("availability") != "Available":
230             continue
231
232         # Building filter (case-insensitive)
233         if building_filter and room.get("building", "").lower() != building_filter.lower():
234             continue
235
236         # Capacity filter (minimum)
237         if min_capacity is not None and int(room.get("capacity", 0)) < min_capacity:
238             continue
239
240         # Time filter: desired_time must be within available_from .. available_until
241         if desired_time is not None:
242             start = room.get("available_from")
243             end = room.get("available_until")
244
245             # If time fields missing, skip (can't prove time availability)
246             if not start or not end:
247                 continue
248
249             try:
250                 start_time = datetime.strptime(start, "%H:%M").time()
251                 end_time = datetime.strptime(end, "%H:%M").time()
252             except ValueError:
253                 continue
254
255             if not (start_time <= desired_time <= end_time):
256                 continue
257
```

Figure 23 - Study room code (section 4).

```
campus_assistant.py 9+ X
campus_assistant.py > ...
257
258     # If all filters pass
259     print(
260         f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
261         f"{bold}BUILDING{reset}: {room.get('building')} | "
262         f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
263         f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} | "
264         f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A'))"
265     )
266     found = True
267
268     if not found:
269         print("No available rooms matched your search criteria.")
270     else:
271         print("Invalid option. Please choose a valid menu number.")
272
```

Figure 24 - Study room code (section 5). These snippets of code compose the algorithm of the study room section of the loop.

Test 6: Input validation



The screenshot displays a code editor window titled 'campus_assistant.py 9+' and a terminal window below it. The code editor shows Python code for a function that checks if all filters pass. The terminal shows the execution of the program, including a welcome message, a list of options, and a prompt for user input. A red box highlights the message 'Invalid input. Please enter a number corresponding to the options.' which appears after the user enters an invalid input.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
257
258 # If all filters pass
259 print(
260     f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
261     f"{bold}BUILDING{reset}: {room.get('building')} | "
262     f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
263     f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} | "
264     f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A'))"
265 )
266 found = True
267
268 if not found:
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random\Desktop\LD4019 - ASSIGNME
:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - ASSIGNMENT -
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
2
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
user enters invalid input
Invalid input. Please enter a number corresponding to the options.
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
█

Figure 25 - This shows the result of the user entering an invalid input.

Test 7: Option 1 - View all study rooms

The screenshot shows a code editor with a file named `campus_assistant.py`. The code is in Python and implements a function to view all study rooms. The code is as follows:

```
257 /
258
259 # If all filters pass
260 print(
261     f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
262     f"{bold}BUILDING{reset}: {room.get('building')} | "
263     f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
264     f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} | "
265     f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A'))"
266 )
267 found = True
268 if not found:
```

The terminal output shows the following sequence of events:

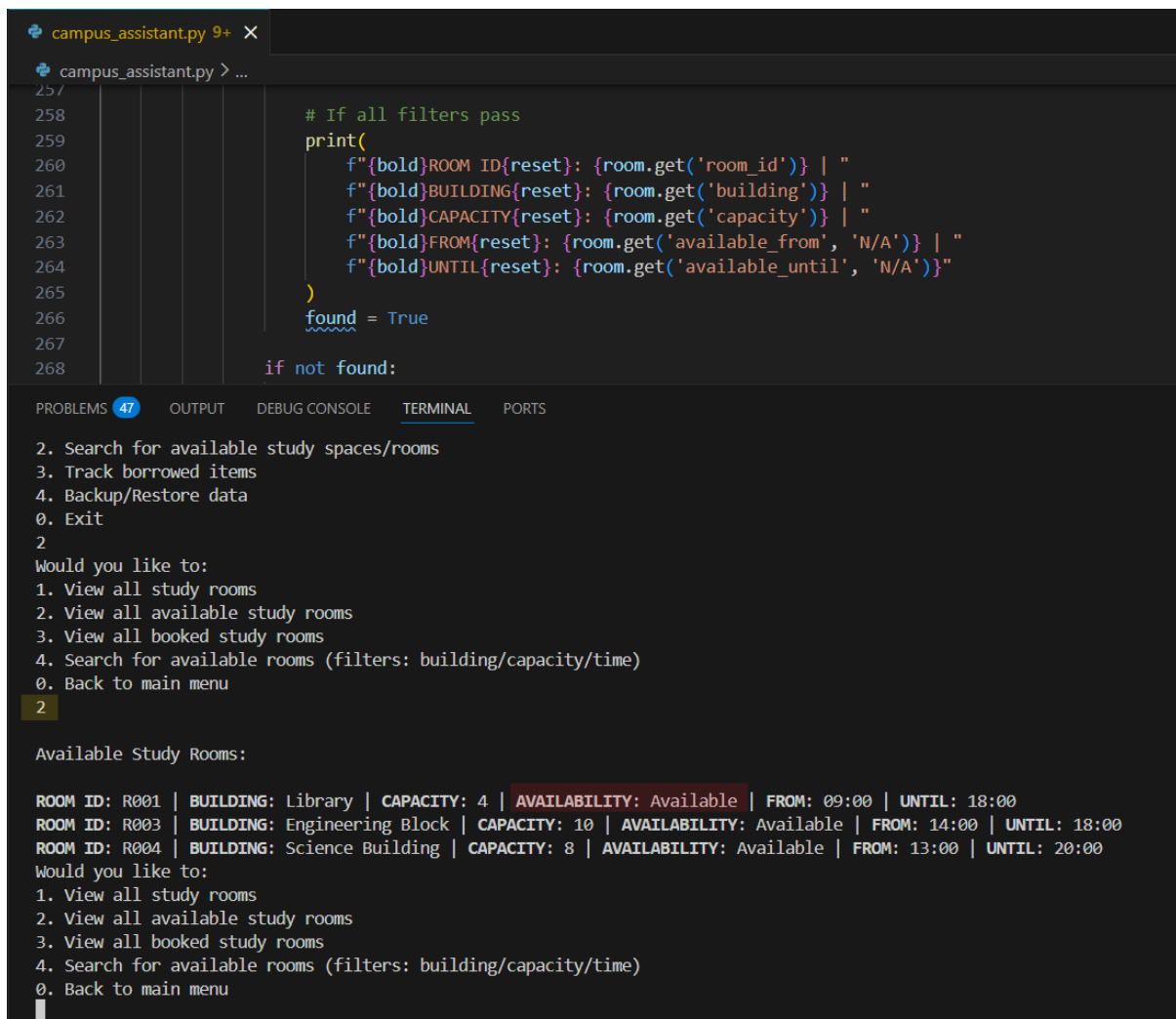
```
PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS
4. Backup/Restore data
0. Exit
2
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
1

All Study Rooms:

ROOM ID: R001 | BUILDING: Library | CAPACITY: 4 | AVAILABILITY: Available | FROM: 09:00 | UNTIL: 18:00
ROOM ID: R002 | BUILDING: Library | CAPACITY: 6 | AVAILABILITY: Booked | FROM: 09:00 | UNTIL: 18:00
ROOM ID: R003 | BUILDING: Engineering Block | CAPACITY: 10 | AVAILABILITY: Available | FROM: 14:00 | UNTIL: 18:00
ROOM ID: R004 | BUILDING: Science Building | CAPACITY: 8 | AVAILABILITY: Available | FROM: 13:00 | UNTIL: 20:00
ROOM ID: R005 | BUILDING: Arts Centre | CAPACITY: 5 | AVAILABILITY: Booked | FROM: 12:00 | UNTIL: 18:00
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
```

Figure 26 - This shows the output of option 1, all study rooms are shown.

Test 8 : Option 2 - View all available study rooms



The screenshot displays a code editor with a file named `campus_assistant.py` and a terminal window below it. The code in the editor shows a function that prints room details if all filters pass, and a `found` variable is set to `True`. The terminal shows the program's output, including a menu where option 2 is selected, leading to a list of available study rooms.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
257
258 # If all filters pass
259 print(
260     f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
261     f"{bold}BUILDING{reset}: {room.get('building')} | "
262     f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
263     f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} | "
264     f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A'))}"
265 )
266 found = True
267
268 if not found:
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

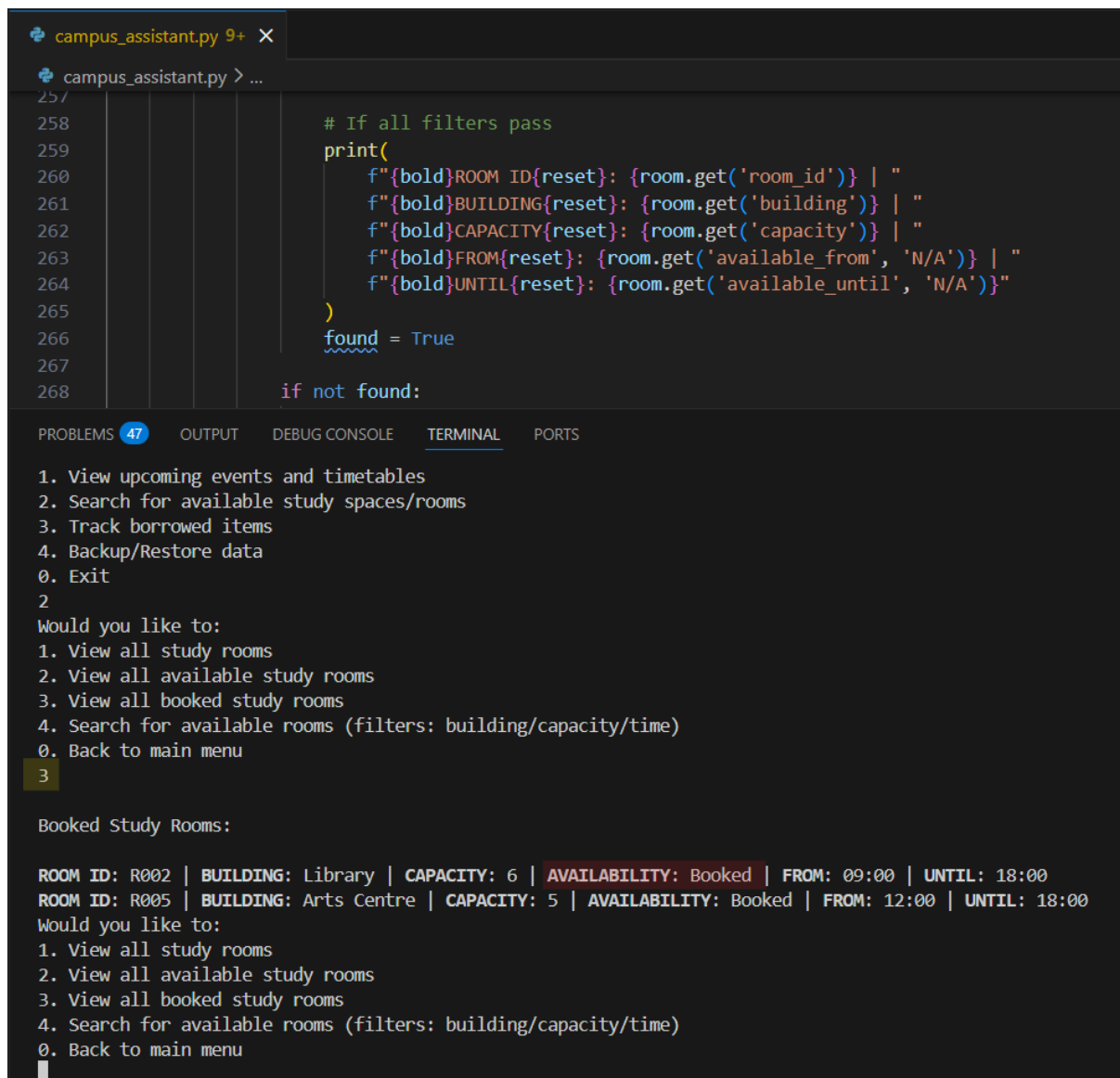
```
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
2
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
2

Available Study Rooms:

ROOM ID: R001 | BUILDING: Library | CAPACITY: 4 | AVAILABILITY: Available | FROM: 09:00 | UNTIL: 18:00
ROOM ID: R003 | BUILDING: Engineering Block | CAPACITY: 10 | AVAILABILITY: Available | FROM: 14:00 | UNTIL: 18:00
ROOM ID: R004 | BUILDING: Science Building | CAPACITY: 8 | AVAILABILITY: Available | FROM: 13:00 | UNTIL: 20:00
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
```

Figure 27 - This shows the output of option 2 within the study room section.

Test 9: Option 3 - View all booked study rooms



The image shows a code editor with a file named `campus_assistant.py`. The code is a Python script for a campus assistant. It includes a menu system and a function to view booked study rooms. The terminal output shows the user selecting option 3, which displays a list of booked study rooms.

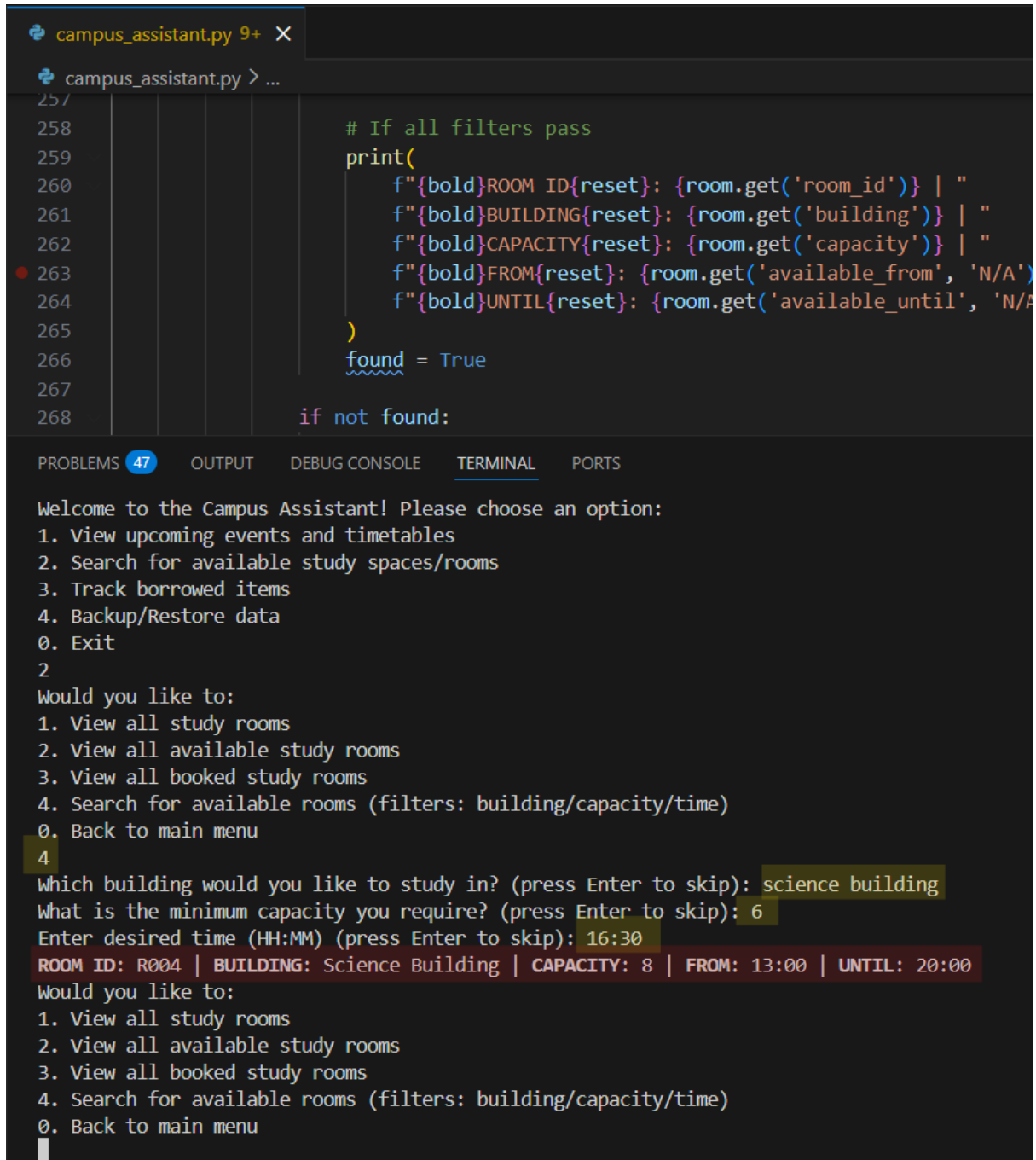
```
campus_assistant.py 9+ X
campus_assistant.py > ...
257
258
259
260
261
262
263
264
265
266
267
268
# If all filters pass
print(
    f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
    f"{bold}BUILDING{reset}: {room.get('building')} | "
    f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
    f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} | "
    f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A'))"
)
found = True
if not found:
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
2
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
3
Booked Study Rooms:
ROOM ID: R002 | BUILDING: Library | CAPACITY: 6 | AVAILABILITY: Booked | FROM: 09:00 | UNTIL: 18:00
ROOM ID: R005 | BUILDING: Arts Centre | CAPACITY: 5 | AVAILABILITY: Booked | FROM: 12:00 | UNTIL: 18:00
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
```

Figure 28 - This shows the output of the third option of the study room options (view booked study rooms).

Test 10: Option 4 - Search for available study rooms by building/capacity/time filter



The image shows a code editor window titled 'campus_assistant.py 9+' and a terminal window below it. The code editor displays Python code for a search function. The terminal shows the output of the program, including a menu, user input, and the resulting search results for a specific room.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
257
258     # If all filters pass
259     print(
260         f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
261         f"{bold}BUILDING{reset}: {room.get('building')} | "
262         f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
263         f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} | "
264         f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A')} | "
265     )
266     found = True
267
268     if not found:
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Welcome to the Campus Assistant! Please choose an option:

1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit

2

Would you like to:

1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu

4

Which building would you like to study in? (press Enter to skip): science building

What is the minimum capacity you require? (press Enter to skip): 6

Enter desired time (HH:MM) (press Enter to skip): 16:30

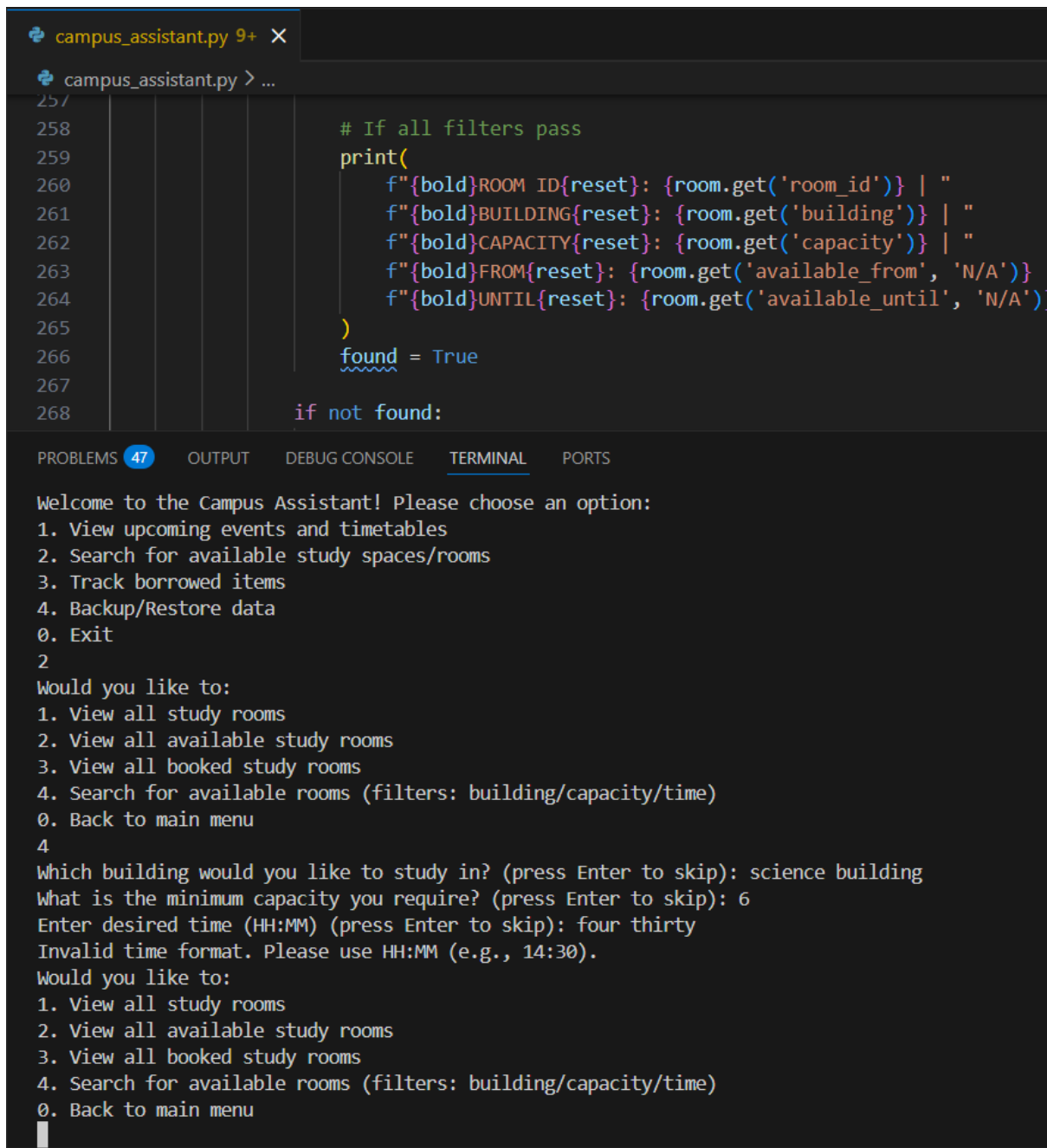
ROOM ID: R004 | BUILDING: Science Building | CAPACITY: 8 | FROM: 13:00 | UNTIL: 20:00

Would you like to:

1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu

Figure 29 - This shows the output of option 4 when all fields match an available room.

Test 11: Error handling of availability time input



```
campus_assistant.py 9+ X
campus_assistant.py > ...
257
258     # If all filters pass
259     print(
260         f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
261         f"{bold}BUILDING{reset}: {room.get('building')} | "
262         f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
263         f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} "
264         f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A')} "
265     )
266     found = True
267
268     if not found:
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Welcome to the Campus Assistant! Please choose an option:

1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit

2

Would you like to:

1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu

4

Which building would you like to study in? (press Enter to skip): science building

What is the minimum capacity you require? (press Enter to skip): 6

Enter desired time (HH:MM) (press Enter to skip): four thirty

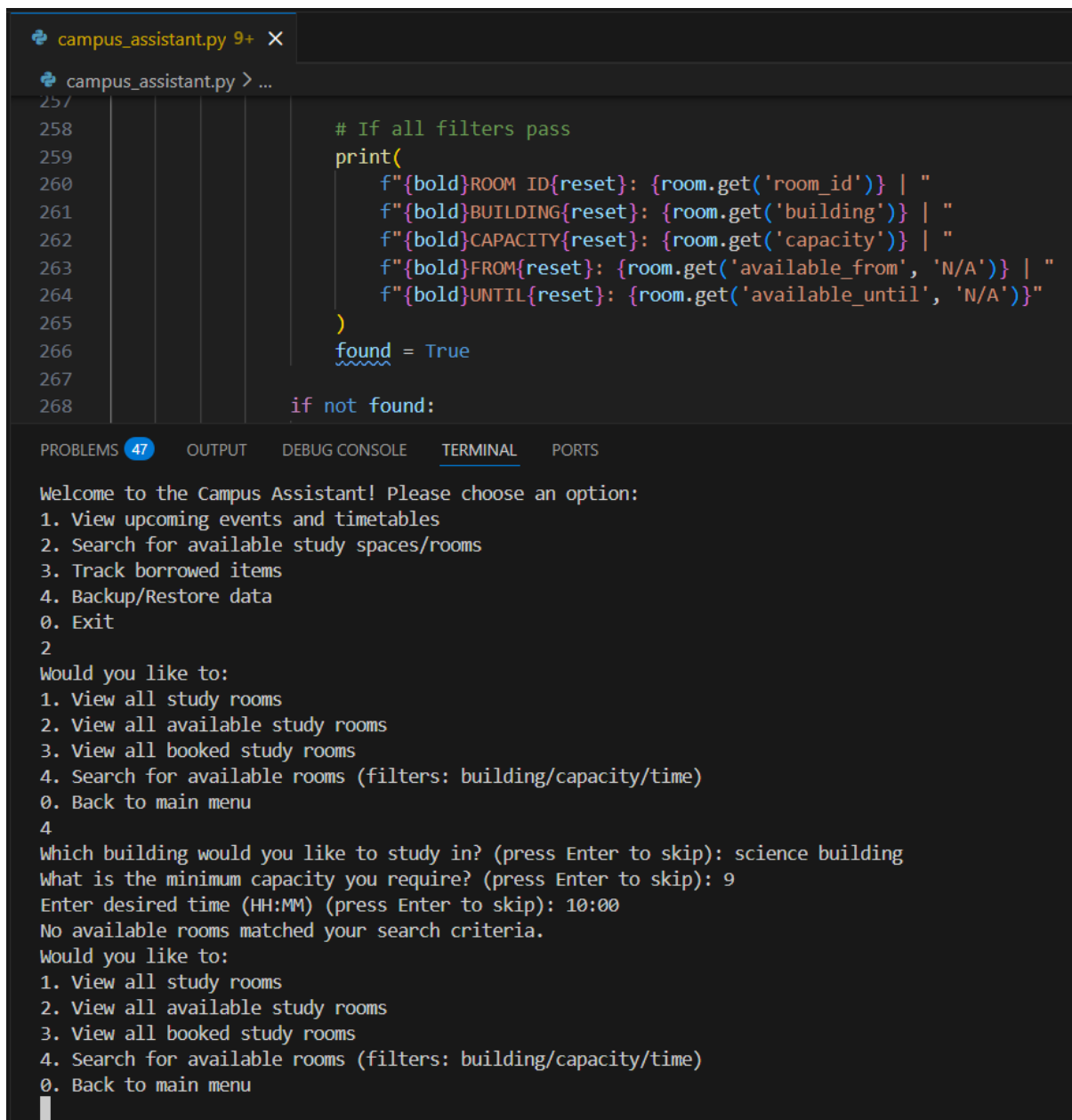
Invalid time format. Please use HH:MM (e.g., 14:30).

Would you like to:

1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu

Figure 30 - This shows error handling of availability times in action.

Test 12: Programme will display an appropriate message if no room scan be found



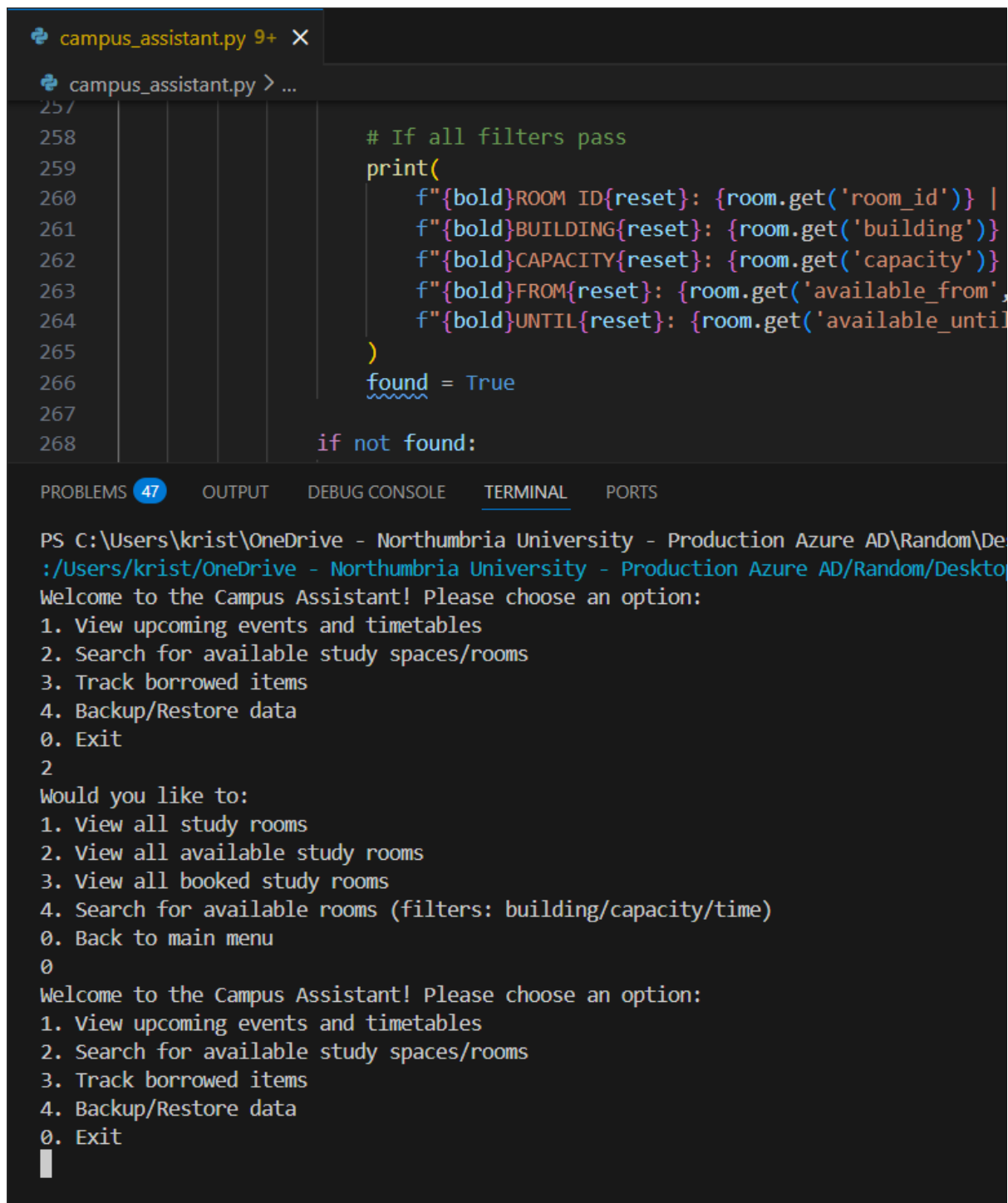
```
campus_assistant.py 9+ X
campus_assistant.py > ...
257
258 # If all filters pass
259 print(
260     f"{bold}ROOM ID{reset}: {room.get('room_id')} | "
261     f"{bold}BUILDING{reset}: {room.get('building')} | "
262     f"{bold}CAPACITY{reset}: {room.get('capacity')} | "
263     f"{bold}FROM{reset}: {room.get('available_from', 'N/A')} | "
264     f"{bold}UNTIL{reset}: {room.get('available_until', 'N/A'))"
265 )
266 found = True
267
268 if not found:
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
2
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
4
Which building would you like to study in? (press Enter to skip): science building
What is the minimum capacity you require? (press Enter to skip): 9
Enter desired time (HH:MM) (press Enter to skip): 10:00
No available rooms matched your search criteria.
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu

Figure 31 - This shows the output of filters that don't match with any available rooms.

Test 13: Option 0 - Back to main menu



The image shows a code editor window with a file named `campus_assistant.py` and a terminal window below it. The code editor shows the following Python code:

```
257/
258     # If all filters pass
259     print(
260         f"{bold}ROOM ID{reset}: {room.get('room_id')} |",
261         f"{bold}BUILDING{reset}: {room.get('building')} |",
262         f"{bold}CAPACITY{reset}: {room.get('capacity')} |",
263         f"{bold}FROM{reset}: {room.get('available_from')} |",
264         f"{bold}UNTIL{reset}: {room.get('available_until')} |",
265     )
266     found = True
267
268 if not found:
```

The terminal window shows the output of the application. It displays the main menu, the user selects option 2, and then the user selects option 0 to return to the main menu. The output is as follows:

```
PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random\Desktop> .\campus_assistant.py
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
2
Would you like to:
1. View all study rooms
2. View all available study rooms
3. View all booked study rooms
4. Search for available rooms (filters: building/capacity/time)
0. Back to main menu
0
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
█
```

Figure 32 – This shows the output of the exit option within the study room options (option 0).

Main Programme Loop: Track, update and search borrowed items

```
campus_assistant.py 9+ X
campus_assistant.py > ...
273 #=====
274 #Option 3: Track, update and search borrowed items
275 #List of dictionaries saved to json
276 #=====
277 elif option == 3:
278     try:
279         borrowed_item_options = int(input(
280             "Would you like to:\n"
281             "1. Add a borrowed item\n"
282             "2. Remove a borrowed item\n"
283             "3. List borrowed items\n"
284             "4. Search for borrowed items\n"
285             "0. Back to main menu\n"
286             ).strip())
287     except ValueError:
288         print("Invalid input. Please enter a number corresponding to the options.")
289         continue
290
291     if borrowed_item_options == 0:
292         continue
293
294     elif borrowed_item_options == 1:
295         print("Please provide the following details:")
296         new_item_id = input("Enter an Item ID: ").strip()
297         new_item_name = input("Enter the name of this new item: ").strip()
298         new_student_id = input("Enter the student ID of the borrower: ").strip()
299         new_student_name = input("Enter the new borrower's name: ").strip()
300
```

Figure 33 - Borrowed items code (section 1).

```
campus_assistant.py 9+ X
campus_assistant.py > ...
301     while True:
302         try:
303             new_borrow_date = input("Enter the borrow date (YYYY-MM-DD): ").strip()
304             datetime.strptime(new_borrow_date, "%Y-%m-%d")
305             break
306         except ValueError:
307             print("Invalid date format. Please use YYYY-MM-DD.")
308     while True:
309         try:
310             new_due_date = input("Enter the due date (YYYY-MM-DD): ").strip()
311             datetime.strptime(new_due_date, "%Y-%m-%d")
312             break
313         except ValueError:
314             print("Invalid date format. Please use YYYY-MM-DD.")
315     while True:
316         new_status = input("Enter the status of this item (Borrowed/Returned): ").strip()
317         if new_status in ["Borrowed", "Returned"]:
318             break
319         else:
320             print("Invalid status. Please enter 'Borrowed' or 'Returned'.")
321
322     new_item = {
323         "item_id": new_item_id,
324         "item_name": new_item_name,
325         "student_id": new_student_id,
326         "student_name": new_student_name,
327         "borrow_date": new_borrow_date,
328         "due_date": new_due_date,
329         "status": new_status
330     }
```

Figure 34 - Borrowed items code (section 2).

```
campus_assistant.py 9+ X
campus_assistant.py > ...
332     if any(item["item_id"] == new_item_id for item in borrowed_items):
333         print("An item with this Item ID already exists. Please use a unique Item ID.")
334         continue
335     borrowed_items.append(new_item)
336     save_json("borrowed_items.json", borrowed_items)
337     print("Borrowed item added successfully.")
338
339     #=====
340     # Remove: borrowed item:
341     # Algorithm: linear search by item_id, remove if found, else notify not found.
342     elif borrowed_item_options == 2:
343         removal = input("Please enter the item ID of the borrowed item you wish to remove: ").strip()
344         if removal == "":
345             print("Invalid input. Please enter a valid Item ID.")
346         else:
347             removed = False
348             for item in borrowed_items:
349                 if item["item_id"] == removal:
350                     borrowed_items.remove(item)
351                     save_json("borrowed_items.json", borrowed_items)
352                     print("Borrowed item removed successfully.")
353                     removed = True
354                     break
355             if not removed:
356                 print("No borrowed item found with that ID.")
357
```

Figure 35 - Borrowed items code (section 3).

```
campus_assistant.py 9+ X
campus_assistant.py > ...
358
359     elif borrowed_item_options == 3:
360         print("List of Borrowed Items:")
361         for item in borrowed_items:
362             print(
363                 f"- {bold}ITEM ID{reset}: {item['item_id']} | "
364                 f"{bold}ITEM NAME{reset}: {item['item_name']} | "
365                 f"{bold}BORROWER NAME{reset}: {item['student_name']} | "
366                 f"{bold}BORROW DATE{reset}: {item['borrow_date']} | "
367                 f"{bold}DUE DATE{reset}: {item['due_date']} | "
368                 f"{bold}STATUS{reset}: {item['status']}\n"
369             )
370
371     #=====
372     # Search for borrowed items
373     #Algorithm: linear search thorough list to find matching item_id, item_name, student_id, student_name
374     elif borrowed_item_options == 4:
375         search = input("Enter Item ID, Student ID, or Student Name to search for: ").strip().lower()
376         if not search:
377             print("Search input cannot be blank.")
378             continue
379         found = False
```

Figure 36 - Borrowed items code (section 4). This code composes the borrowed items section of the loop.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
380
381     for item in borrowed_items:
382         item_id = str(item.get("item_id", "")).lower()
383         student_id = str(item.get("student_id", "")).lower()
384         student_name = str(item.get("student_name", "")).lower()
385         item_name = str(item.get("item_name", "")).lower()
386
387         if (
388             search == item_id
389             or search == student_id
390             or search in student_name
391             or search in item_name
392         ):
393             print(
394                 f"- {bold}ITEM ID{reset}: {item['item_id']} | "
395                 f"{bold}ITEM NAME{reset}: {item['item_name']} | "
396                 f"{bold}BORROWER NAME{reset}: {item['student_name']} | "
397                 f"{bold}STUDENT ID{reset}: {item['student_id']} | "
398                 f"{bold}BORROW DATE{reset}: {item['borrow_date']} | "
399                 f"{bold}DUE DATE{reset}: {item['due_date']} | "
400                 f"{bold}STATUS{reset}: {item['status']}"
401             )
402             found = True
403             break
404
405         if not found:
406             print("No borrowed items match the filters provided.")
407
```

Figure 37 - Borrowed items code (section 5). This code composes the borrowed items section of the loop.

Test 14: Error message will display when invalid input is entered



```
campus_assistant.py 9+ X
campus_assistant.py > ...
343     removal = input("Please enter the item ID of the borrowed item you want to remove: ")
344     if removal == "":
345         print("Invalid input. Please enter a valid Item ID.")
346     else:
347         removed = False
348         for item in borrowed_items:
349             if item["item_id"] == removal:
350                 borrowed_items.remove(item)
351                 save_json("borrowed_items.json", borrowed_items)
352                 print("Borrowed item removed successfully.")
353                 removed = True
354                 break
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random\Desktop\LD40> .\campus_assistant.py
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
3
Would you like to:
1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu
user enters a random input
Invalid input. Please enter a number corresponding to the options.
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
█

Figure 38 - This shows that an error message will occur if the user does not choose one of the prompted options.

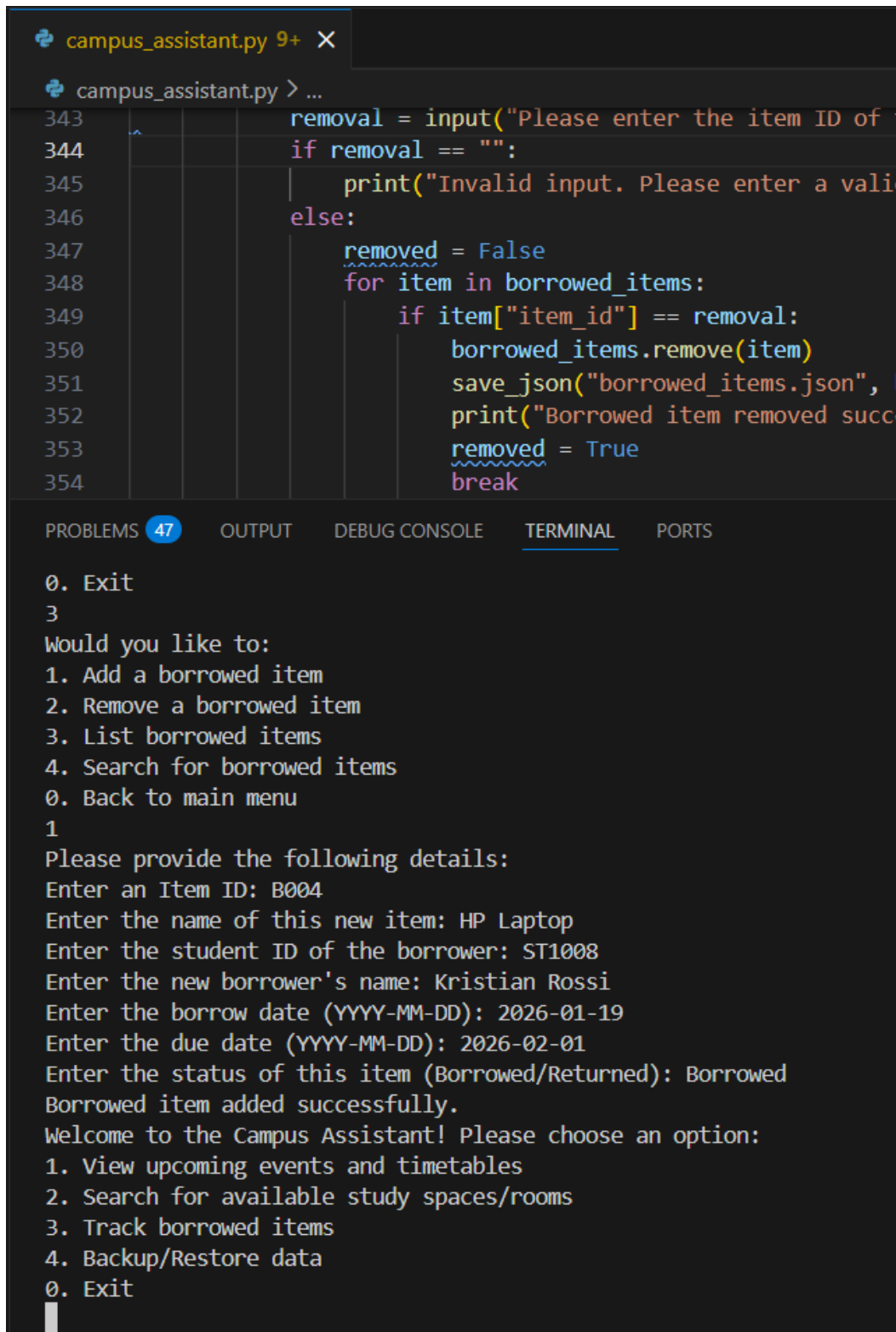
Option 1: Add a borrowed item



```
campus_assistant.py 5  borrowed_items.json X
{} borrowed_items.json > ...
1  [
2      {
3          "item_id": "B001",
4          "item_name": "Laptop",
5          "student_id": "ST1001",
6          "student_name": "Alice Green",
7          "borrow_date": "2025-03-01",
8          "due_date": "2025-03-10",
9          "status": "Borrowed"
10     },
11     {
12         "item_id": "B002",
13         "item_name": "Microscope",
14         "student_id": "ST1002",
15         "student_name": "James Brown",
16         "borrow_date": "2025-03-03",
17         "due_date": "2025-03-12",
18         "status": "Borrowed"
19     },
20     {
21         "item_id": "B003",
22         "item_name": "Football Kit",
23         "student_id": "ST1003",
24         "student_name": "Sarah Lee",
25         "borrow_date": "2025-03-05",
26         "due_date": "2025-03-15",
27         "status": "Returned"
28     }
29 ]
```

Figure 39 - This is the data within the json file before any modifications.

Test 15: All necessary fields will be asked for when adding a new borrowed item



The screenshot displays a code editor window titled 'campus_assistant.py 9+ X' and a terminal window below it. The code editor shows Python code for removing a borrowed item. The terminal shows the execution of the program, where option 1 is selected to add a new borrowed item. The user is prompted to enter various details, which are then entered and confirmed.

```
campus_assistant.py > ...
343 removal = input("Please enter the item ID of t
344 if removal == "":
345     print("Invalid input. Please enter a valid
346 else:
347     removed = False
348     for item in borrowed_items:
349         if item["item_id"] == removal:
350             borrowed_items.remove(item)
351             save_json("borrowed_items.json", l
352             print("Borrowed item removed succo
353             removed = True
354             break
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
0. Exit
3
Would you like to:
1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu
1
Please provide the following details:
Enter an Item ID: B004
Enter the name of this new item: HP Laptop
Enter the student ID of the borrower: ST1008
Enter the new borrower's name: Kristian Rossi
Enter the borrow date (YYYY-MM-DD): 2026-01-19
Enter the due date (YYYY-MM-DD): 2026-02-01
Enter the status of this item (Borrowed/Returned): Borrowed
Borrowed item added successfully.
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
```

Figure 40 - This shows the process of option 1 within the borrowed items section (add a borrowed item).

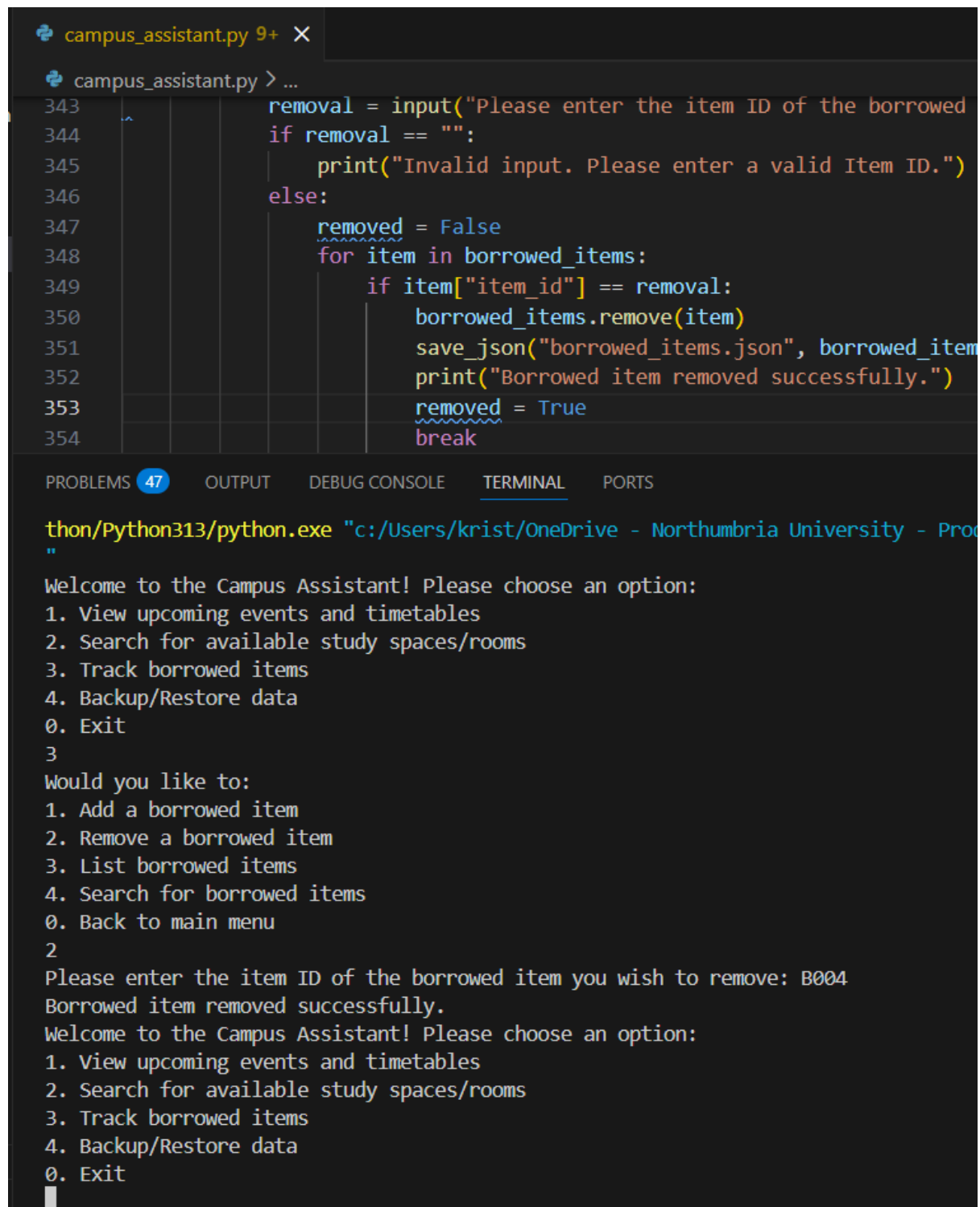
Test 16: New borrowed item will be saved to the JSON file



```
campus_assistant.py 9+  {} borrowed_items.json X
{} borrowed_items.json > ...
2      {
3          "student_id": "ST1001",
4          "student_name": "Alice Green",
5          "borrow_date": "2025-03-01",
6          "due_date": "2025-03-10",
7          "status": "Borrowed"
8      },
9      {
10         "item_id": "B002",
11         "item_name": "Microscope",
12         "student_id": "ST1002",
13         "student_name": "James Brown",
14         "borrow_date": "2025-03-03",
15         "due_date": "2025-03-12",
16         "status": "Borrowed"
17     },
18     {
19         "item_id": "B003",
20         "item_name": "Football Kit",
21         "student_id": "ST1003",
22         "student_name": "Sarah Lee",
23         "borrow_date": "2025-03-05",
24         "due_date": "2025-03-15",
25         "status": "Returned"
26     },
27     {
28         "item_id": "B004",
29         "item_name": "HP Laptop",
30         "student_id": "ST1008",
31         "student_name": "Kristian Rossi",
32         "borrow_date": "2026-01-19",
33         "due_date": "2026-02-01",
34         "status": "Borrowed"
35     }
36 ]
```

Figure 41 - This shows that the new borrowed item was successfully appended to the borrowed_items.json file.

Test 17: Option 2 - Remove a borrowed item



The screenshot displays a code editor window titled 'campus_assistant.py 9+' and a terminal window below it. The code in the editor shows a function for removing a borrowed item. It prompts the user for an item ID, checks if it's valid, and if so, removes it from a list and saves the updated list to a JSON file.

```
343 removal = input("Please enter the item ID of the borrowed\n344 if removal == \"\":\n345     print("Invalid input. Please enter a valid Item ID.")\n346 else:\n347     removed = False\n348     for item in borrowed_items:\n349         if item["item_id"] == removal:\n350             borrowed_items.remove(item)\n351             save_json("borrowed_items.json", borrowed_items)\n352             print("Borrowed item removed successfully.")\n353             removed = True\n354             break
```

The terminal window shows the execution of the program. It starts with a welcome message and a list of options. The user selects option 2, 'Remove a borrowed item'. The program then prompts for an item ID, and the user enters 'B004'. The program confirms the removal and returns to the main menu.

```
thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumbria University - Pro\n\nWelcome to the Campus Assistant! Please choose an option:\n1. View upcoming events and timetables\n2. Search for available study spaces/rooms\n3. Track borrowed items\n4. Backup/Restore data\n0. Exit\n3\nWould you like to:\n1. Add a borrowed item\n2. Remove a borrowed item\n3. List borrowed items\n4. Search for borrowed items\n0. Back to main menu\n2\nPlease enter the item ID of the borrowed item you wish to remove: B004\nBorrowed item removed successfully.\nWelcome to the Campus Assistant! Please choose an option:\n1. View upcoming events and timetables\n2. Search for available study spaces/rooms\n3. Track borrowed items\n4. Backup/Restore data\n0. Exit
```

Figure 42 - This shows the process of option 2 (removing a borrowed item).

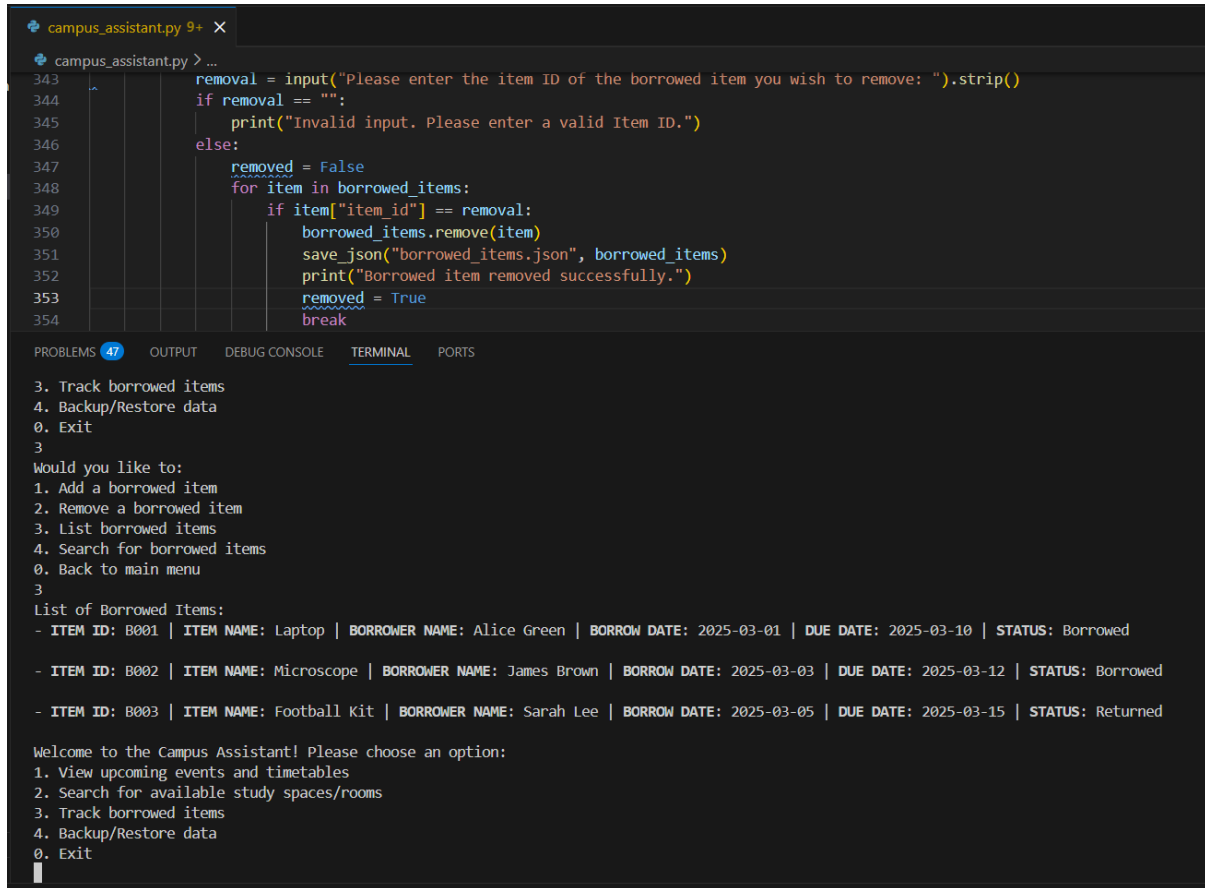
Test 18: the JSON file will be updated when an item is removed



```
campus_assistant.py 9+  borrowed_items.json X
{} borrowed_items.json > ...
1  [
2  {
3      "item_id": "B001",
4      "item_name": "Laptop",
5      "student_id": "ST1001",
6      "student_name": "Alice Green",
7      "borrow_date": "2025-03-01",
8      "due_date": "2025-03-10",
9      "status": "Borrowed"
10 },
11 {
12     "item_id": "B002",
13     "item_name": "Microscope",
14     "student_id": "ST1002",
15     "student_name": "James Brown",
16     "borrow_date": "2025-03-03",
17     "due_date": "2025-03-12",
18     "status": "Borrowed"
19 },
20 {
21     "item_id": "B003",
22     "item_name": "Football Kit",
23     "student_id": "ST1003",
24     "student_name": "Sarah Lee",
25     "borrow_date": "2025-03-05",
26     "due_date": "2025-03-15",
27     "status": "Returned"
28 }
29 ]
```

Figure 43 - This shows the added item has now successfully been removed from the borrowed_items.json file.

Test 19: Option 3 – The terminal will list borrowed items



The screenshot shows a code editor with a file named `campus_assistant.py` and a terminal window below it. The code in the editor implements a function to remove a borrowed item. The terminal shows the program's output, including a menu of options and a list of borrowed items.

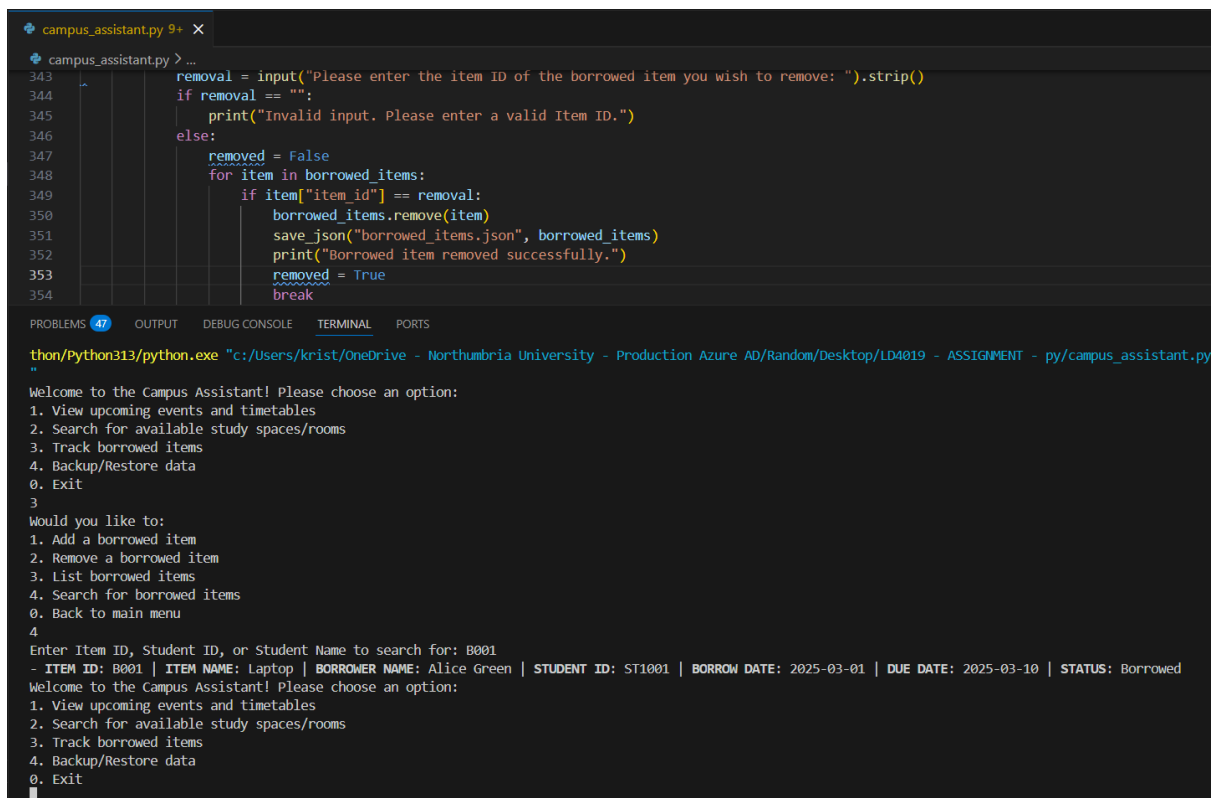
```
campus_assistant.py 9+ X
campus_assistant.py > ...
343 removal = input("Please enter the item ID of the borrowed item you wish to remove: ").strip()
344 if removal == "":
345     print("Invalid input. Please enter a valid Item ID.")
346 else:
347     removed = False
348     for item in borrowed_items:
349         if item["item_id"] == removal:
350             borrowed_items.remove(item)
351             save_json("borrowed_items.json", borrowed_items)
352             print("Borrowed item removed successfully.")
353             removed = True
354             break
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

3. Track borrowed items
4. Backup/Restore data
0. Exit
3
Would you like to:
1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu
3
List of Borrowed Items:
- ITEM ID: B001 | ITEM NAME: Laptop | BORROWER NAME: Alice Green | BORROW DATE: 2025-03-01 | DUE DATE: 2025-03-10 | STATUS: Borrowed
- ITEM ID: B002 | ITEM NAME: Microscope | BORROWER NAME: James Brown | BORROW DATE: 2025-03-03 | DUE DATE: 2025-03-12 | STATUS: Borrowed
- ITEM ID: B003 | ITEM NAME: Football Kit | BORROWER NAME: Sarah Lee | BORROW DATE: 2025-03-05 | DUE DATE: 2025-03-15 | STATUS: Returned
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit

Figure 44 - This shows the output of option 3 within the borrowed items section (list borrowed items).

Test 20: Option 4 – Users can search for items by item ID



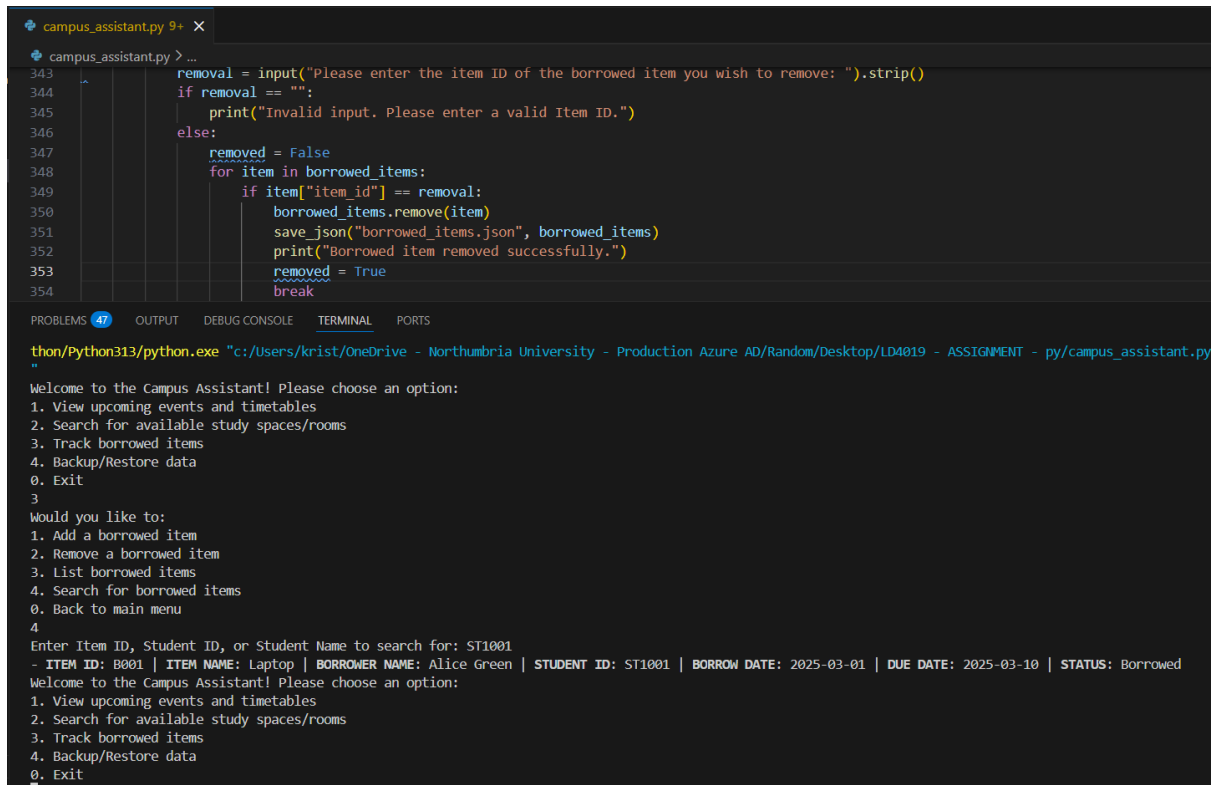
The screenshot displays a code editor with a file named `campus_assistant.py` and a terminal window below it. The code in the editor shows a function to remove a borrowed item by its ID. The terminal shows the program's execution, including a menu, user input, and the successful removal of an item.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
343 removal = input("Please enter the item ID of the borrowed item you wish to remove: ").strip()
344 if removal == "":
345     print("Invalid input. Please enter a valid Item ID.")
346 else:
347     removed = False
348     for item in borrowed_items:
349         if item["item_id"] == removal:
350             borrowed_items.remove(item)
351             save_json("borrowed_items.json", borrowed_items)
352             print("Borrowed item removed successfully.")
353             removed = True
354             break

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS
thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - ASSIGNMENT - py/campus_assistant.py"
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
3
Would you like to:
1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu
4
Enter Item ID, Student ID, or Student Name to search for: B001
- ITEM ID: B001 | ITEM NAME: Laptop | BORROWER NAME: Alice Green | STUDENT ID: ST1001 | BORROW DATE: 2025-03-01 | DUE DATE: 2025-03-10 | STATUS: Borrowed
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
```

Figure 45 - This shows filtering by item ID is working.

Test 21: Option 4 - Users can search for items by student ID



The screenshot shows a code editor with a file named `campus_assistant.py`. The code is in Python and implements a function to remove a borrowed item. The terminal output shows the program's execution, including a menu of options and the successful removal of an item.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
343 removal = input("Please enter the item ID of the borrowed item you wish to remove: ").strip()
344 if removal == "":
345     print("Invalid input. Please enter a valid Item ID.")
346 else:
347     removed = False
348     for item in borrowed_items:
349         if item["item_id"] == removal:
350             borrowed_items.remove(item)
351             save_json("borrowed_items.json", borrowed_items)
352             print("Borrowed item removed successfully.")
353             removed = True
354             break
```

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - ASSIGNMENT - py/campus_assistant.py"

Welcome to the Campus Assistant! Please choose an option:

1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit

3

Would you like to:

1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu

4

Enter Item ID, Student ID, or Student Name to search for: ST1001

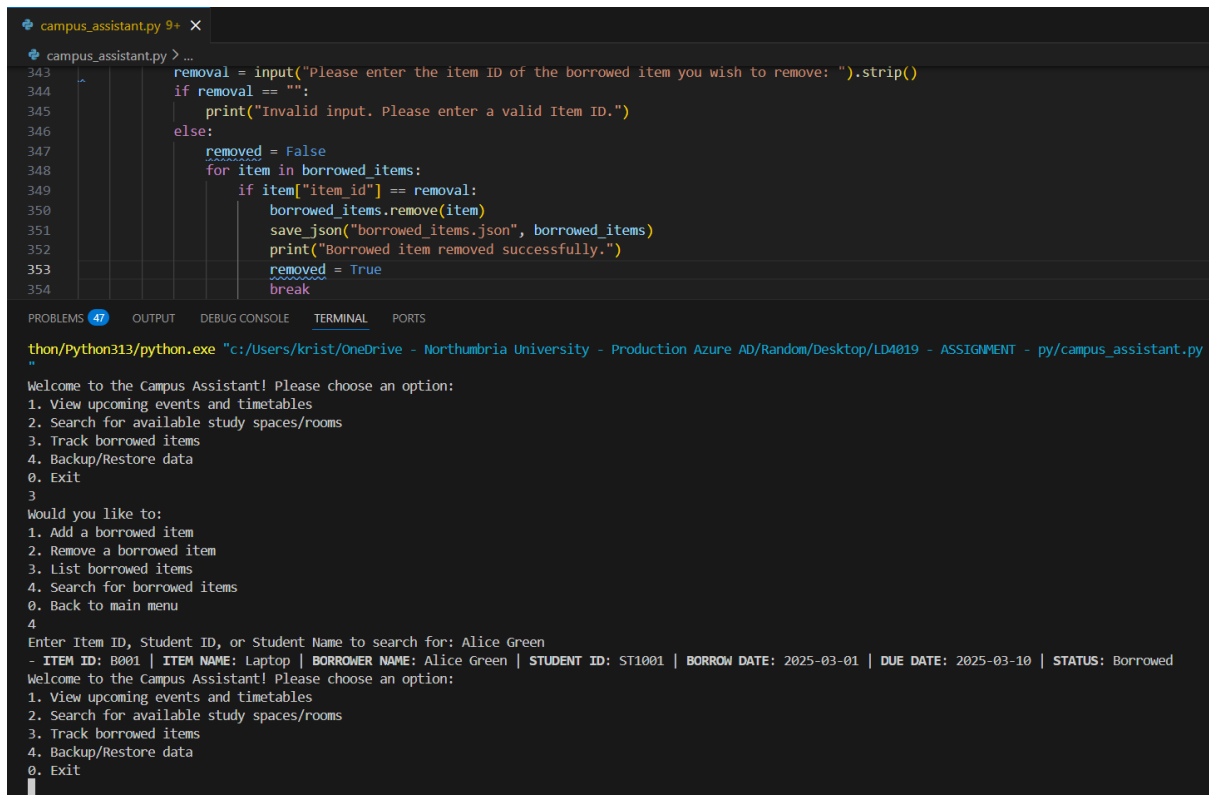
- ITEM ID: 8001 | ITEM NAME: Laptop | BORROWER NAME: Alice Green | STUDENT ID: ST1001 | BORROW DATE: 2025-03-01 | DUE DATE: 2025-03-10 | STATUS: Borrowed

Welcome to the Campus Assistant! Please choose an option:

1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit

Figure 46 - This shows filtering by student ID is working.

Test 22: Option 4 - Users can search for items by student name



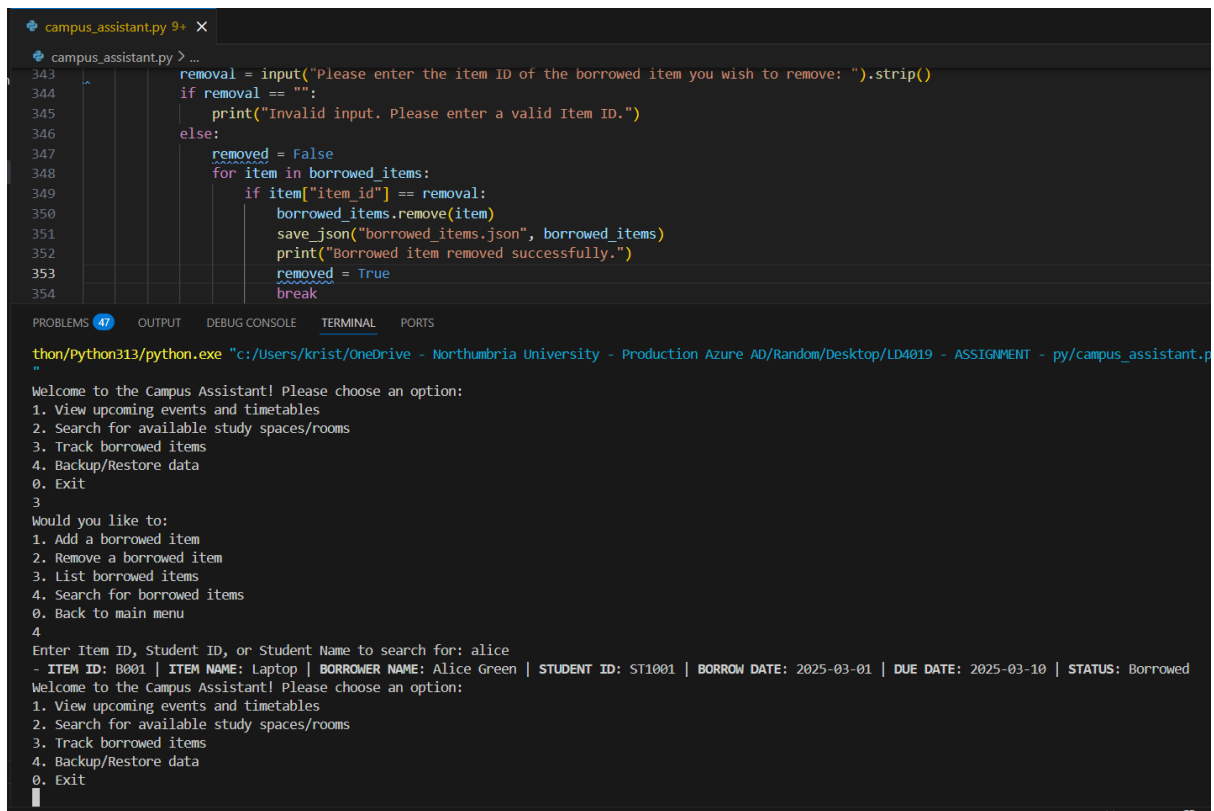
The screenshot displays a code editor window titled 'campus_assistant.py 9+ X' and a terminal window below it. The code in the editor shows a function to remove a borrowed item by its ID. The terminal shows the application's main menu, where option 4 (Search for borrowed items) is selected. The user enters 'Alice Green' as the search criteria, and the application displays a list of borrowed items filtered by that student name.

```
campus_assistant.py 9+ X
campus_assistant.py > ...
343 removal = input("Please enter the item ID of the borrowed item you wish to remove: ").strip()
344 if removal == "":
345     print("Invalid input. Please enter a valid Item ID.")
346 else:
347     removed = False
348     for item in borrowed_items:
349         if item["item_id"] == removal:
350             borrowed_items.remove(item)
351             save_json("borrowed_items.json", borrowed_items)
352             print("Borrowed item removed successfully.")
353             removed = True
354             break

PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS
thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - ASSIGNMENT - py/campus_assistant.py"
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
3
Would you like to:
1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu
4
Enter Item ID, Student ID, or Student Name to search for: Alice Green
- ITEM ID: B001 | ITEM NAME: Laptop | BORROWER NAME: Alice Green | STUDENT ID: ST1001 | BORROW DATE: 2025-03-01 | DUE DATE: 2025-03-10 | STATUS: Borrowed
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
```

Figure 47 - This shows filtering by student name is working.

Test 23: Option 4 – Partial filtering works



The screenshot displays a code editor with a file named `campus_assistant.py`. The code implements a function to remove a borrowed item by its ID. It prompts the user for an item ID, checks if it's empty, and if not, iterates through a list of borrowed items to find a match. Upon finding a match, it removes the item, saves the updated list to a JSON file, and prints a success message. The terminal output shows the application's main menu, where option 4 (Backup/Restore data) is selected. It then prompts for an item ID, student ID, or student name to search for. The user enters 'alice', and the application displays a list of items, including a laptop borrowed by Alice Green. The terminal output also shows the application's main menu again, where option 4 is selected.

```
343 removal = input("Please enter the item ID of the borrowed item you wish to remove: ").strip()
344 if removal == "":
345     print("Invalid input. Please enter a valid Item ID.")
346 else:
347     removed = False
348     for item in borrowed_items:
349         if item["item_id"] == removal:
350             borrowed_items.remove(item)
351             save_json("borrowed_items.json", borrowed_items)
352             print("Borrowed item removed successfully.")
353             removed = True
354             break
```

thom/python313/python.exe "c:/Users/krist/OneDrive - Northumbria University - Production Azure AD/Random/Desktop/LD4019 - ASSIGNMENT - py/campus_assistant.p

Welcome to the Campus Assistant! Please choose an option:

1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit

3

Would you like to:

1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu

4

Enter Item ID, Student ID, or Student Name to search for: alice

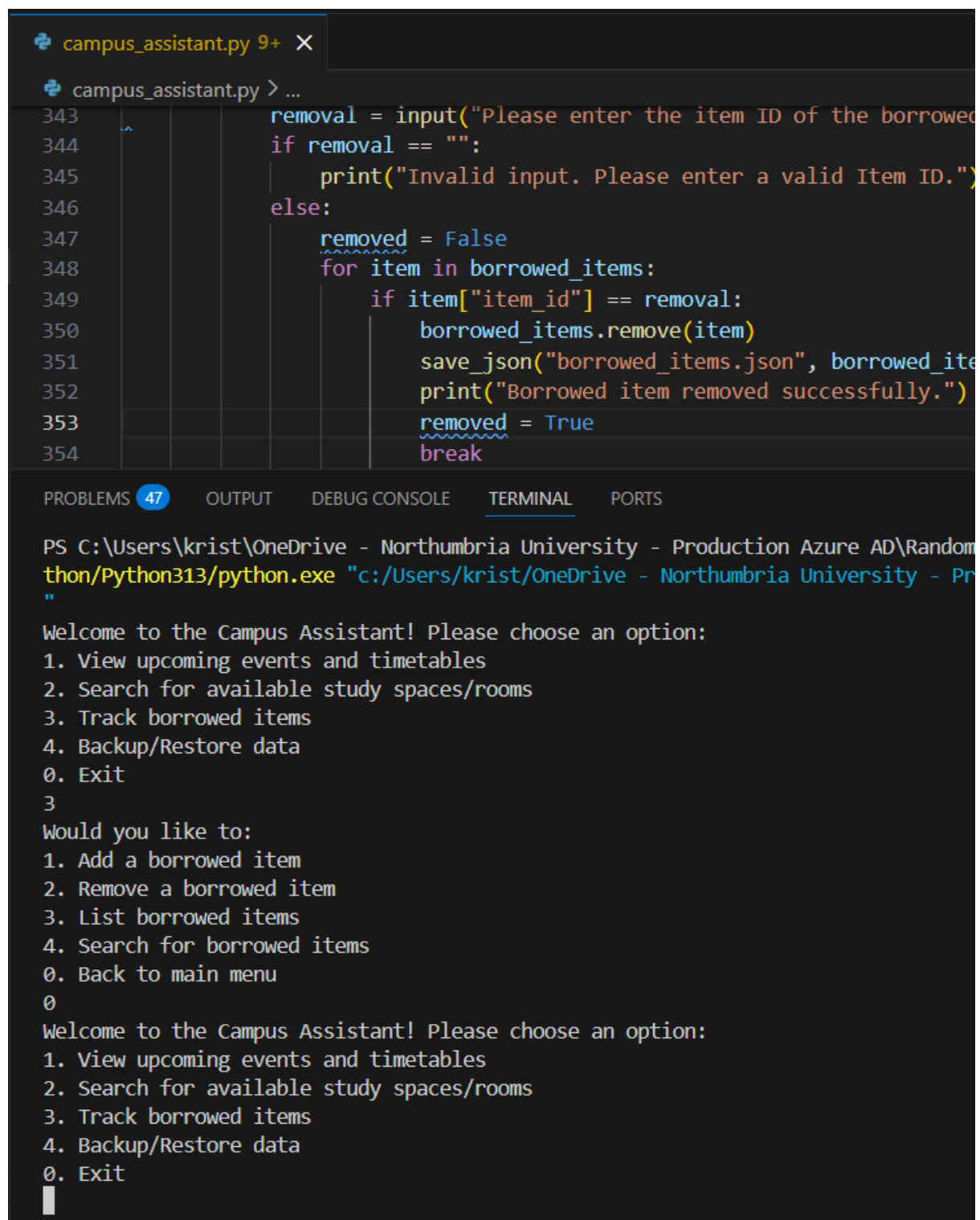
- ITEM ID: B001 | ITEM NAME: Laptop | BORROWER NAME: Alice Green | STUDENT ID: ST1001 | BORROW DATE: 2025-03-01 | DUE DATE: 2025-03-10 | STATUS: Borrowed

Welcome to the Campus Assistant! Please choose an option:

1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit

Figure 48 - This shows partial filtering works.

Test 24: Option 0: Exit



The screenshot shows a code editor with a file named `campus_assistant.py` and a terminal window below it. The code in the editor is as follows:

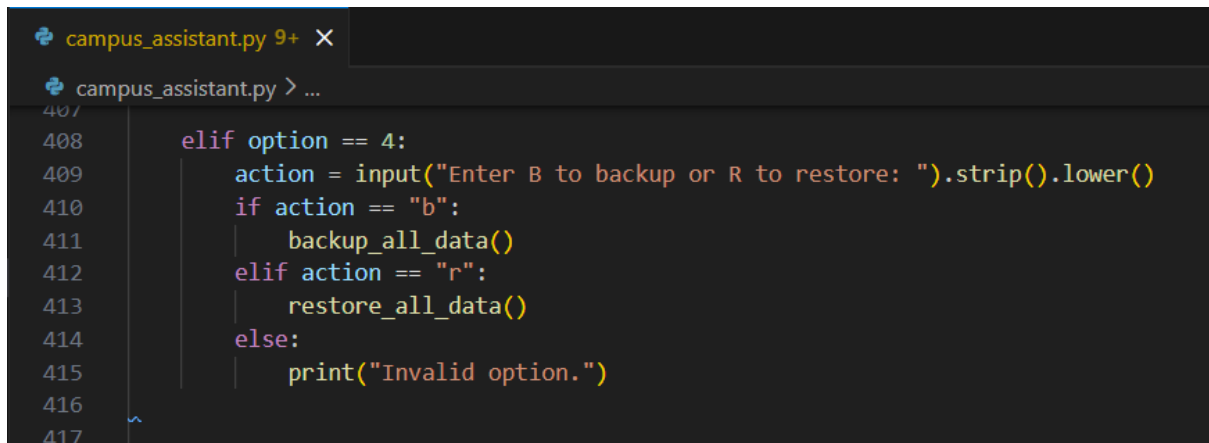
```
343 removal = input("Please enter the item ID of the borrowed
344 if removal == "":
345     print("Invalid input. Please enter a valid Item ID.")
346 else:
347     removed = False
348     for item in borrowed_items:
349         if item["item_id"] == removal:
350             borrowed_items.remove(item)
351             save_json("borrowed_items.json", borrowed_items)
352             print("Borrowed item removed successfully.")
353             removed = True
354             break
```

The terminal window shows the command prompt at `PS C:\Users\krist\OneDrive - Northumbria University - Production Azure AD\Random` and the command `thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumbria University - Pr"` being executed. The output of the program is as follows:

```
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
3
Would you like to:
1. Add a borrowed item
2. Remove a borrowed item
3. List borrowed items
4. Search for borrowed items
0. Back to main menu
0
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
```

Figure 49 - This shows the result of option 0 (Back to main menu).

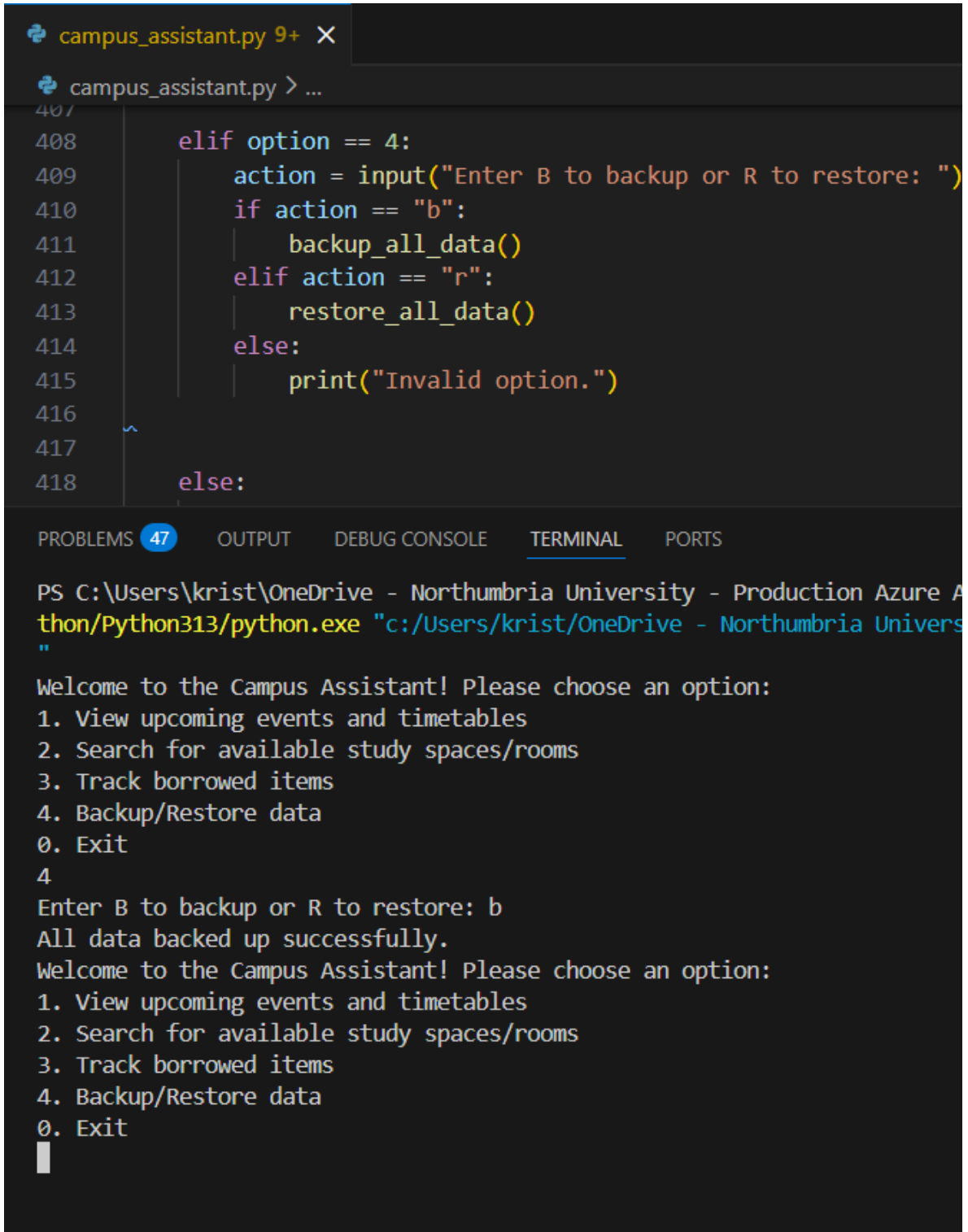
Main Programme Loop: Backup/Restore Data

A screenshot of a code editor window titled 'campus_assistant.py 9+ X'. The editor shows a Python script with line numbers 407 to 417. The code is a snippet from a larger program, specifically the backup/restore menu option. It starts with an 'elif' statement for 'option == 4'. Inside this block, it prompts the user to 'Enter B to backup or R to restore: ', strips whitespace, and converts the input to lowercase. It then uses an 'if' statement to check if the action is 'b' (backup) or 'r' (restore), calling 'backup_all_data()' or 'restore_all_data()' respectively. If the input is neither, it prints 'Invalid option.'.

```
407  
408     elif option == 4:  
409         action = input("Enter B to backup or R to restore: ").strip().lower()  
410         if action == "b":  
411             backup_all_data()  
412         elif action == "r":  
413             restore_all_data()  
414         else:  
415             print("Invalid option.")  
416  
417
```

Figure 50 - This shows the block of code for backup/restore menu option.

Test 25: Data can be backup successfully (confirmation the backup function runs)



```
campus_assistant.py 9+ X
campus_assistant.py > ...
407
408     elif option == 4:
409         action = input("Enter B to backup or R to restore: ")
410         if action == "b":
411             backup_all_data()
412         elif action == "r":
413             restore_all_data()
414         else:
415             print("Invalid option.")
416
417
418     else:
```

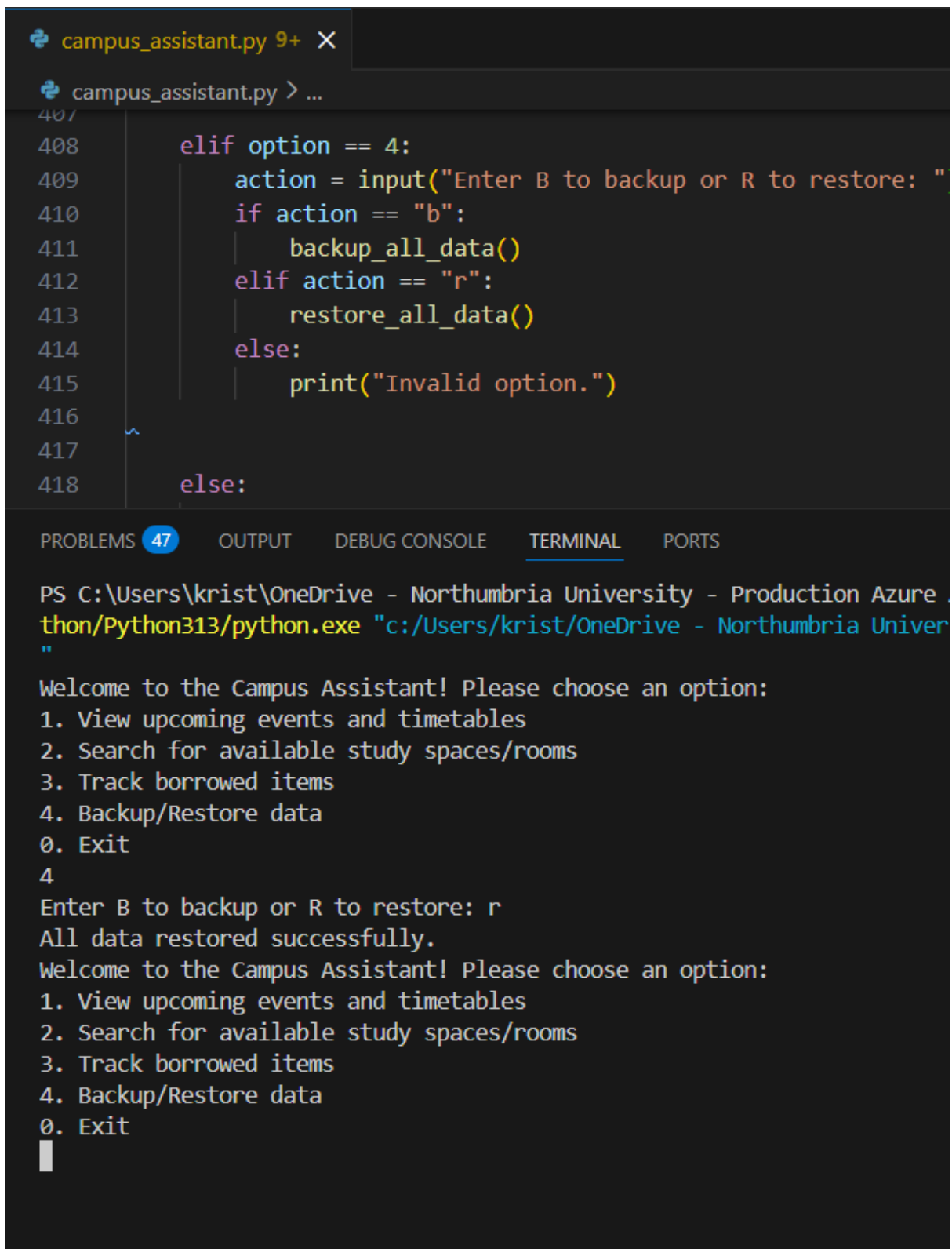
PROBLEMS 47 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\krist\OneDrive - Northumbria University - Production Azure A
thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumbria Univers
"

Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
4
Enter B to backup or R to restore: b
All data backed up successfully.
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
█

Figure 51 - This message shows the output of the restore data option being selected.

Test 26: Data can be restored successfully (the restoring data function will run)



The screenshot displays a code editor window with a file named `campus_assistant.py`. The code is a Python script for a 'Campus Assistant' application. It features a menu-driven interface where users can view events, search for study spaces, track borrowed items, backup/restore data, or exit. The code is as follows:

```
407
408     elif option == 4:
409         action = input("Enter B to backup or R to restore: ")
410         if action == "b":
411             backup_all_data()
412         elif action == "r":
413             restore_all_data()
414         else:
415             print("Invalid option.")
416
417
418     else:
```

Below the code editor, the 'TERMINAL' tab is active, showing the execution of the program. The terminal output is as follows:

```
PS C:\Users\krist\OneDrive - Northumbria University - Production Azure
thon/Python313/python.exe "c:/Users/krist/OneDrive - Northumbria Univer
"

Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
4
Enter B to backup or R to restore: r
All data restored successfully.
Welcome to the Campus Assistant! Please choose an option:
1. View upcoming events and timetables
2. Search for available study spaces/rooms
3. Track borrowed items
4. Backup/Restore data
0. Exit
█
```

Figure 52 - This shows the output of the restore option being selected by the user.

Validation of loop from start to end:

The “while True” line at the very top of the loop and the “else:” line at the very bottom of the loop validate two things. The “while True” line ensures that the loop will run indefinitely and will continuously prompt the main menu to the user until they exit it. The “else:” line ensures that invalid menu selections are handled gracefully by displaying an error message instead of terminating the program.

This section proves that all the Campus Assistant requirements have been met successfully.

Critical Reflection

Ethical Considerations

The main ethical concern in this project is student data privacy. The system stores identifiable student data such as their names and IDs when tracking borrowed items. Even though data is stored in a local JSON file rather than a more insecure way like transmitting it over a network, it still constitutes as data that needs to be protected to maintain student trust by keeping their data safe. Ethical risks are mitigated within the scope of the assignment by limiting data stored to only what is strictly necessary and avoiding the use of personal student information.

Social Considerations

A key social consideration that arose during development was the system's prioritisation of accessibility and usability through a simple menu-driven interface with clear prompts and error messages. This reduces user confusion and supports users with basic technical skills. However, the reliance on a terminal-based interface limits accessibility for some users, particularly those with visual impairments or low familiarity with command-line systems. A graphical or web-based interface would improve inclusivity in a real deployment.

Security Considerations

The key security concern that arose was the lack of authentication. Any user running the program can modify or remove records if they know an item ID, which could lead to misuse or inaccurate data. This weakness was accepted due to the limited scope of the assignment, but it highlights the need for user authentication and access control if this system were to be used in a real-world scenario. This also links to the ethical consideration of student privacy, no control of modification or accessibility allows anyone to view identifiable student data, contradicting GDPR requirements.

References

1. GeeksforGeeks (2019). Python Data Types. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/python/python-data-types/>.
2. John Sturtz (2018). Dictionaries in Python. [online] Realpython.com. Available at: <https://realpython.com/python-dicts/>.
3. Ali, M. (2018). Python JSON Data: A Guide With Examples. [online] Datacamp.com. Available at: <https://www.datacamp.com/tutorial/json-data-python>.
4. Mwit, D. (2025). Python Try-Except Tutorial: Best Practices and Real-World Examples. [online] Datacamp.com. Available at: <https://www.datacamp.com/tutorial/python-try-except>.
5. Codecademy (2025). Pseudocode and Flowchart: Complete Beginner's Guide. [online] Codecademy. Available at: <https://www.codecademy.com/article/pseudocode-and-flowchart-complete-beginners-guide>.
6. GeeksforGeeks (2017). Loops in Python For, While and Nested Loops. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/python/loops-in-python/>.
7. BBC Bitesize. (n.d.). Condition-controlled loops - using WHILE - Iteration in programming - KS3 Computer Science Revision. [online]

Available at:

<https://www.bbc.co.uk/bitesize/guides/z3khpv4/revision/5>.

8. London, A.J., Razin, Y.S., Borenstein, J., Eslami, M., Perkins, R. and Robinette, P. (2023). Ethical Issues in Near-Future Socially Supportive Smart Assistants for Older Adults. *IEEE Transactions on Technology and Society*, pp.1–1. doi:<https://doi.org/10.1109/tts.2023.3237124>.
9. Sommer, D. (2024). Insights - Understanding the smart campus revolution - Drees & Sommer. [online] Drees & Sommer. Available at: <https://www.dreso.uk/insights/understanding-the-smart-campus-revolution> [Accessed 20 Jan. 2026].
10. Absolute Security. (2025). Security Systems For The Education Sector - Absolute Security. [online] Available at: https://www.absolutesecurity.co.uk/security-systems-for-the-education-sector/?gad_source=1&gad_campaignid=225714039&gbraid=0AAAAADskjbm46YP79qMvdFttwoPcjiURn&gclid=CjwKCAiA7LzLBhAgEiwAjMWzCPXlK6CBZO2--wvj0egvwwqPhwnEdWdli_1lbyrFcU5bDpSlL6PFtBoCoK0QAvD_BwE [Accessed 20 Jan. 2026].